When Are Agent Execution Edits Safe? An Exact Theory of Fork, Restore, and Merge

***Abstract*—Agent runtimes can Fork an execution, Restore a checkpoint, or Merge branches without restarting a task. We call these operations *execution edits* because they change which parts of the workflow will run next. An edit cannot undo a tool call that has already been authorized or released. A careless edit can therefore perform the same action twice, drop work that the workflow still requires, or overlap with a call that began before the edit. We ask when an execution edit can be implemented safely. Our answer preserves every outcome that was required before the edit. Each such outcome must remain required, already have a recorded completion, or be explicitly removed by the same policy authority that required it. Existing Agent recovery systems implement particular edits, while existing synthesis methods assume that the possible executions and completion conditions are already known. Neither derives the full safety requirement of a requested edit from the execution record. We present an exact theory and an executable checker. Given a registered workflow, its authenticated record, and a requested edit, the checker constructs every complete execution allowed by the workflow and policy. It removes an execution whenever one of its partial executions would leave a still-required outcome with no safe way to finish. If nothing remains, the checker returns a finite proof that no safe implementation exists. Otherwise, the remaining executions define the most permissive runtime monitor that enforces the edit. For all six registered execution edits and registered extensions, our main theorem proves that the checker accepts exactly when such a safe implementation exists. The same theorem proves atomic installation despite racing calls, identifies the four kinds of recorded information required by any exact checker, and relates the installed runtime to an ideal atomic machine. Every supported event preserves the security invariant, so the theorem applies again after every finite reachable execution. Five kernel-checked Lean modules, 81 executable tests, and bounded performance measurements validate the finite checker, critical examples, and executable artifact.**

I. Introduction

Tool-using Agent runtimes no longer follow only one linear plan. They let an Agent Fork an execution into alternatives, Restore an earlier checkpoint, and Merge branches [1], [2], [3], [4], [5], [6]. These operations support exploration, recovery, and reuse without restarting the task. We call them *execution edits* because they change which parts of the workflow will run next.

An execution edit does not undo earlier tool calls. In our running procurement example, the Agent saves checkpoint k before approval. The approval later uses its one-use permission and leaves a completion record. Restoring k creates another approval call in the workflow, but it must not authorize the same approval again. A concurrent Merge can cause a similar error by selecting supplier L after a supplier-R payment has already been authorized. An edit can therefore restore the saved program state and still be unsafe.

The problem is that a checkpoint contains only part of the information needed to decide whether an edit is safe. The controller must also know four things: which outcomes remain required, which call positions refer to the same protected action, which actions have already been authorized, and which runtime version governs a call that overlaps the edit. Each can change the correct answer. The controller cannot recover all four from equal workspaces, an execution trace alone, a receipt log alone, or a replacement workflow proposed by the Agent.

Existing approaches cover parts of this problem but do not derive its complete safety condition. Agent systems provide branching, staged effects, transactions, or recovery checks [7], [8], [9], [10], [11], [12], [13], [14]. Control and synthesis choose behavior after a model of possible executions and desired final states has been supplied [15], [16], [17], [18]. Neither line of work derives from a recorded execution everything that a particular edit must preserve before it takes effect.

Our key idea is that an execution edit must preserve both past actions and still-required outcomes. Every outcome required before the edit must remain required, already have a recorded completion, or be explicitly removed by the same policy authority that required it. The Agent selects only a registered edit. The trusted controller derives the edited workflow and the requirements it inherits from the recorded execution. The Agent therefore cannot obtain a favorable answer by submitting a replacement workflow that hides inconvenient facts.

This idea creates three technical problems. First, calls copied by Fork or Restore may refer to the same protected action, so the controller must distinguish a new action from reuse of an earlier call and return value. Second, after every allowed partial execution, every outcome that is still required and still possible must retain a safe way to finish. The checker must enforce this condition without forbidding additional safe behavior. Third, the answer must take effect atomically with respect to calls that overlap the edit or a crash.

We give an exact checker for execution edits. For a requested edit, the controller uses the registered workflow, recorded calls and edits, completion records, pending calls, policy, and current monitor version to construct every allowed completed execution. It removes an execution whenever one of its partial executions would leave a still-required outcome with no safe completion. If no execution remains, the controller returns a finite proof that no safe implementation exists. Otherwise, the remaining executions define the most permissive safe monitor. Before the edit takes effect, the runtime checks the current recorded state again and atomically replaces the old monitor. Every

overlapping call is therefore governed entirely by either the old monitor or the new one. Section II follows this process through one procurement execution containing Fork, Merge, two races, and Restore. After Restore, the repeated approval call reuses the earlier completion record instead of authorizing the approval again.

a) Contributions: We contribute one end-to-end theorem connecting the original Agent workflow, the requested execution edit, the exact checker, and the installed runtime. For all six registered execution edits and registered extensions, a safe implementation exists exactly when the checker leaves a nonempty set of executions, its monitor can be installed, and an ideal atomic machine permits the same edit. The theorem also proves that all four kinds of recorded information used by the checker are necessary and that the concrete runtime has the same visible behavior as the ideal machine (sections III to VI). Every supported event preserves the security invariant, so the theorem applies again after every finite reachable execution. Five kernel-checked Lean modules, 81 executable tests, and bounded performance measurements validate the finite checker, critical examples, and executable artifact.

II. A Procurement Edit, End to End

Consider a procurement Agent authorized to place one order. The Agent reserves the budget and saves checkpoint k before approval. The approval then uses its one-use permission and leaves a completion record. The Agent next requests a Fork between suppliers L and R. The workflow allows two outcomes: order from L or order from R. In either outcome, payment must precede shipment:

$$\begin{aligned} o_L &: \; x_L^p \prec x_L^s, \\ o_R &: \; x_R^p \prec x_R^s, \end{aligned} \tag{1}$$

where x_i^p and x_i^s are payment and shipment occurrences. An occurrence identifies one call position in the workflow. An effect identity d identifies the protected action and one-use permission behind that position.

The two shipment calls share one effect identity:

$$q_{\mathrm{cell}}(x_L^s) = q_{\mathrm{cell}}(x_R^s) = d_s.$$

The two payment calls instead have different effect identities d_L^p and d_R^p. A later Restore of k creates a new approval occurrence x_a', but both approval occurrences refer to the same effect identity d_a.

The receipt log Δ records newly authorized effects in order. The notation $\mathsf{lab}(\Delta)$ keeps only their permission labels. Budget reservation contributes r, approval contributes a, the two payments contribute p_L and p_R, and shipment contributes s. At the Fork request, $\mathsf{lab}(\Delta) = ra$. The policy allows prefixes of rap_Ls and rap_Rs, but never both payments or a payment before approval. Although the supplier outcomes are exclusive, both must still have a safe way to finish when the Fork is checked.

a) How the checker handles the Fork: The Agent submits only $u = \mathsf{ForkChoice}(b, \sigma)$. The trusted controller, not the Agent, constructs the edited workflow from the registered edit schema. It reads the workflow, authenticated execution record, receipt log, ordered dispatch queue, and policy, written $(\kappa, H, \Delta, \mathsf{out}, \mathcal{A})$. From these records it derives the edited workflow, its required outcomes, and the effect identity behind every call position.

Write M_o for the policy-safe completions of outcome o. In this example, each outcome has one allowed call order:

$$\begin{aligned} z_L &= \mathsf{fresh}(x_L^p, d_L^p)\mathsf{fresh}(x_L^s, d_s), \\ z_R &= \mathsf{fresh}(x_R^p, d_R^p)\mathsf{fresh}(x_R^s, d_s). \end{aligned}$$

Thus $M_{o_L} = \{z_L\}$, $M_{o_R} = \{z_R\}$, and $\widehat{B}^\dagger = \{(o_L, z_L), (o_R, z_R)\}$, with $W^\dagger$ their prefix closure. The checker builds monitor $\mathsf{Can}(W^\dagger, \pi_2\widehat{B}^\dagger)$, binds the answer to this source snapshot, and atomically installs the monitor and replacement authorization tokens under a new monitor version. The installed monitor allows only these partial executions. For each protected call, fresh means that the effect identity receives its first receipt, while alias means that another call position reuses the existing receipt. If policy excludes rap_Rs, then $M_{o_R} = \varnothing$, and the Fork is rejected because pruning supplier R would change its meaning. Rejection returns a ranked certificate explaining which completions were removed and why, rather than letting the Agent weaken the requested edit. Section IV gives a harder case where each outcome initially has a safe completion, but every possible implementation eventually reaches a partial execution after which some still-possible outcome cannot finish safely.

b) Adding new rules without changing current requirements: A registered extension $\mathsf{Extend}(\xi)$ can add compensation schemas and policy entries while the procurement execution remains active. The protected-call and policy authorities must both sign ξ. The extension preserves every current outcome, call position, effect identity, receipt, checkpoint, cursor, dispatch-queue entry, and causal edge. It only appends new schema and registry rows and enlarges the policy. The controller then rebuilds the monitor and atomically changes its version. A later edit may use the new schema, but the Agent cannot change an earlier call or remove a currently required outcome.

A *policy domain* is the set of protected actions governed by one policy, one receipt log, and one monitor that is replaced atomically.

c) The fresh *race:* After the shared approval and before either payment, the Agent requests $\mathsf{MergeSelect}(g, L, \sigma)$. If the payment call x_R^p from the old supplier-R branch is authorized first, it selects R and changes $\mathsf{lab}(\Delta)$ from ra to rap_R. Selecting L must therefore be re-derived and rejected. If the Merge installation occurs first, it closes the old monitor version and gives every current call a new authorization token before any later x_R^p can be authorized. Leaving the old monitor version active would allow both payments.

d) The alias *race:* Now consider the Restore that keeps both continuations in figure 1. Copying checkpoint k creates a new approval occurrence x_a' with a new monitor entry

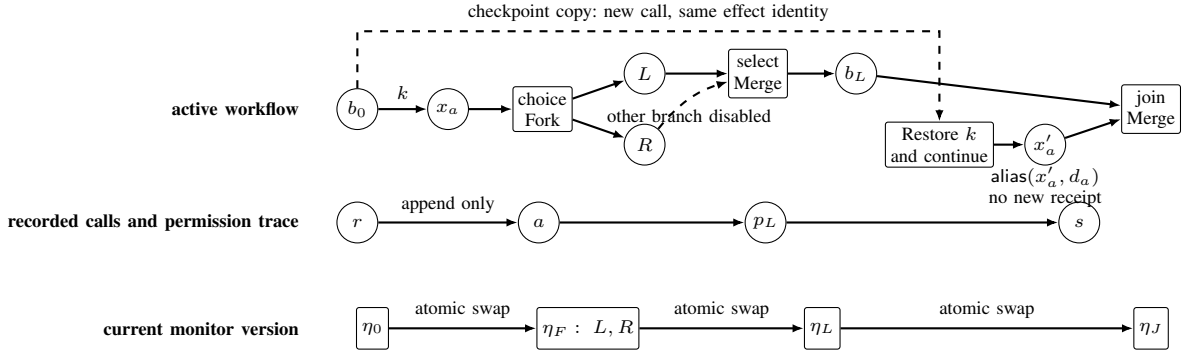

Fig. 1. Fork, Restore, and Merge change the active workflow but do not erase recorded calls. Restore creates a new call position while preserving its authenticated effect identity. The atomic installation point must order both a new authorization (fresh) and reuse of an earlier completion record (alias) against the monitor-version change.

and authorization token, but it retains effect identity d_a. Because d_a already has a receipt, executing x'_a is an alias: the restored branch advances past approval without creating another receipt. A concurrent join or replacement computed before that progress is stale even though the receipt log has not changed. The installation check must therefore compare the ordered occurrence record χ as well as the receipt log. For this RestoreLive request, the checker either installs the derived monitor under a new version or returns a ranked rejection certificate.

The example exposes the whole problem. Workspace equality does not reveal which branches remain active, and receipt equality does not reveal alias progress. Effect-identity equality alone does not reveal which outcomes the edit must preserve. A replacement monitor proposed by the Agent cannot certify any of these facts. Section III therefore derives the edited workflow from the registered schema and authenticated execution record.

III. Model and Edit Derivation

The model separates unrestricted internal Agent reasoning from tool calls whose effects pass through a trusted runtime monitor. Such a call must identify its registered position in the workflow, present an authorization token, and submit a concrete tool request. The monitor decides whether the call may use a new authorization, reuse an earlier completion record, or make no progress. Each modeled state covers one policy domain. Domains with disjoint protected actions, completion logs, and policies compose by product. Every state snapshot is finite, although an execution may contain an unbounded number of snapshots. The definitions below first describe the required outcomes and then the authenticated record needed to preserve them across edits.

A. Workflow outcomes and call order

a) Typed runtime vocabulary: We capitalize *Agent* for the untrusted principal requesting execution edits. An *occurrence* x identifies one position for a protected tool call in the workflow. An *effect identity* d identifies the one protected action and one-use permission behind that position. The first authorized use of an effect identity creates a receipt. After Fork or Restore, several occurrences may refer to the same effect identity. A monitor entry j belongs to one monitor version η. An authorization token h binds an occurrence to its monitor entry, effect identity, scope, and normalized tool request m°. *Lineage* is the certified ancestry preserved when an edit carries or copies a workflow region. T, Δ, χ denote the active workflow, receipt log, and ordered record of resolved occurrences. An edit takes effect at an atomic installation point, which also replaces the policy domain's monitor version. Registry ρ authenticates each occurrence's monitor entry and token. It maps the occurrence to $q_{\mathrm{cell}}(x) = d$, immutable normalized request $\mathsf{inv}(d) = m^\circ$, and permission label $\ell(d)$.

A pomset $p = (X_p, \prec_p)$ is a finite set of occurrences with a strict partial order. Its linearizations $\mathsf{Lin}(p)$ are the words containing each member of X_p once and respecting $\prec_p$. An *indexed workflow contract* $\mathcal{C} = (O, \{p_o\}_{o \in O})$ is a finite nonempty family of pomsets. Each index o names one allowed completion outcome. Two outcomes remain distinct even if the calls still required for them later become identical. Write $X_{\mathcal{C}} = \bigcup_{o \in O} X_{p_o}$ and $\mathsf{CL}(\mathcal{C}) = \bigcup_{o \in O} \mathsf{Lin}(p_o)$. We require the runtime to distinguish completion from continuation.

$$v, w \in \mathsf{CL}(\mathcal{C}) \ \wedge \ v \preceq_{\mathrm{pref}} w \quad \Longrightarrow \quad v = w.$$

This condition permits incomparable traces and different outcomes with the same trace, so it does not require deterministic outcomes. It requires a registered occurrence whenever continuing after a shared partial execution performs more work. An otherwise silent choice between stopping and continuing lies outside the interface. A deployment may represent such a choice with an authenticated terminal occurrence handled by the controller, without claiming that a remote action occurred.

Contracts use the three partial constructors below. Each constructor requires occurrence-disjoint operands and is defined

only when its result remains completion-observable.

$$\begin{aligned} O_{\mathcal{C}\oplus\mathcal{D}} &= (\{L\}\times O_{\mathcal{C}}) \uplus (\{R\}\times O_{\mathcal{D}}),\\ p_{(L,o)} &= p_o, \qquad p_{(R,v)} = p_v,\\ O_{\mathcal{C}\otimes\mathcal{D}} &= O_{\mathcal{C}}\times O_{\mathcal{D}},\\ p_{(o,v)} &= p_o \uplus p_v,\\ O_{\mathcal{C}\triangleright\mathcal{D}} &= O_{\mathcal{C}}\times O_{\mathcal{D}},\\ p_{(o,v)} &= p_o \triangleright p_v. \end{aligned} \tag{2}$$

Before adding a registered or copied operand, the edit derivation deterministically renames its occurrences and extends ρ while retaining their authenticated effect identities. Well-formedness already makes carried operands disjoint. In equation (2), $\uplus$ adds no cross-order and $\triangleright$ orders every event of p before every event of q, while choice uses a tagged union of outcome indices and parallel and sequence use Cartesian products. Consequently, choice retains each arm's full indexed family, parallel pairs outcomes while permitting every causal linearization, and sequence adds a barrier. For raw prefix w, $\mathcal{C}/w$ retains each index whose pomset has a linearization prefixed by w and removes w's occurrences and incident order, and it is undefined if none remains. This operational "remaining contract" preserves outcome and occurrence identity.

Lemma 1 (Observable completion)**.** *Every defined residual $\mathcal{C}/w$ is completion-observable, and either every retained pomset is empty or no retained pomset is empty.*

Proof. If a residual suffix v strictly prefixes v', then wv strictly prefixes wv' in $\mathsf{CL}(\mathcal{C})$. If one retained pomset is empty and another is nonempty, choose a nonempty linearization v of the latter. Then w strictly prefixes wv. Both contradict completion observability. □

B. Authenticated execution record

1) Recorded structure and active workflow: The authenticated execution record is

$$H = (\omega, G, T, K, \Sigma_\zeta, \rho, \beta, \chi, \nu, \eta). \tag{3}$$

ω identifies the policy domain, and η identifies its current monitor version. G is a finite append-only record of workflow versions. Its node identities, base contracts, and typed edit records never change. Each continuation node b carries an immutable registered base contract Γ_b. Its branch cursor χ_b records which occurrences on that continuation have completed. The immutable records form a directed acyclic graph (DAG) whose edges record typed Fork, Restore, and Merge operations. T is the active workflow.

$$T ::= b{:}\mathcal{C} \mid T \oplus^s_g T \mid T \parallel_g T \mid \mathsf{jbar}(T,T) \mid T;T.$$

$s \in \{\mathsf{open}, L, R\}$ records an unresolved choice or the arm that made durable progress, and g groups exclusive or jointly active siblings. $[\![T]\!]$ uses both arms of an open choice and only the selected arm thereafter. It interprets parallel and jbar with $\otimes$, and sequence with $\triangleright$. The unaddressed jbar marks a joined group and prevents another Merge.

T is complete when every pomset in $[\![T]\!]$ is empty. Lemma 1 excludes a residual containing both empty and nonempty pomsets. Set $\mathsf{heads}(b{:}\mathcal{C}) = \{b\}$. Open choice exposes both arms, selected choice its winner, parallel and jbar both arms, and sequence its left operand until completion and then its right. Completing an occurrence updates the work remaining at its leaf without deleting the recorded structure. Because sequence exposure tests complete, it retains completed left-outcome indices, so an isolated completed leaf remains addressable for later Fork or Restore. Joining removes g immediately but keeps its continuation behind a barrier until both arms complete. An occurrence is enabled when minimal in some pomset of $[\![T]\!]$, consistent with sequence exposure by observable completion. Occurrence disjointness and unique context decomposition give every enabled x a unique exposed leaf $\mathsf{leaf}_T(x) = b$. For $a \in \{\mathsf{fresh}(x,d), \mathsf{alias}(x,d)\}$, define the paired execution update

$$\mathsf{HStep}((G,T),a) = \big(G \,\|\, (b,a),\, T/(x,a)\big), \qquad b = \mathsf{leaf}_T(x),$$

where $G\|(b,a)$ appends the progress record (b,a), and $T/(x,a)$ residualizes the leaf and records any choice selection without storing a second cursor. Write $\mathsf{HRes}((G,T),r)$ for iterating this update over a resolved word r.

$\mathsf{addr}(T)$ follows the same recursion over branch and group identifiers. It includes exposed groups before descending into their active arms and only the exposed sequence operand, excluding inactive alternatives and unexposed operands.

K maps checkpoint identifiers to immutable earlier continuation records. Write $\mathcal{C}_k = \mathsf{contract}(K(k))$ for the residual contract stored by checkpoint k. Σ_ζ is a versioned immutable registry of typed edit schemas. ρ contains the authenticated occurrence-to-monitor-entry/effect-identity map, authorization tokens, lineage, scope, normalized tool requests, and request digests. For each current occurrence x, binding $\beta(x) = (j,\eta)$ names its monitor entry and version. χ is the ordered append-only record of resolved occurrences, including fresh and alias bindings, and ν is the record version. Because χ records authorization and workflow progress rather than remote completion, a return value may still await settlement.

$\mathsf{Save}(k,b)$ is an explicit checkpoint transition, not a seventh execution edit, and requires fresh k and addressable b. A trusted transaction stores an authenticated copy of continuation b's remaining contract $\mathcal{C}_b$, completed-occurrence cursor χ_b, and relevant registry rows. Formally, it appends $(\omega, b, \mathcal{C}_b, \chi_b, \rho|_{X_{\mathcal{C}_b}}, \zeta)$ and its fingerprint to K. The checkpoint check requires $\mathcal{C}_b = \Gamma_b/\mathsf{raw}(\chi_b)$, and the transaction advances ν without changing the active workflow, receipts, bindings, monitor versions, or permission trace. Both machines take this τ-transition, which makes a concurrent installation stale because installation authenticates H. Restore accepts only records created by this rule.

2) Well-formedness:

Definition 1 (Well-formed history)**.** *H is well formed when the following groups hold.*

1) Structure. *G is acyclic, and continuation and group identifiers are globally unique across the recorded DAG and current workflow. Every addressable identifier has a unique context decomposition, and every head of T is a current maximal node. Every checkpoint names an immutable node coordinate and a value-snapshotted residual/cursor pair derived from G.*

2) Identity and progress. *Occurrence and monitor-entry identifiers are unique, while occurrences mapped to the same effect identity agree on the call-authorizing service, scope, normalized request, digest, and lineage. Resolved occurrences are downward closed in their pomset, consistent with every choice status, and absent from the remaining workflow.*

3) Current workflow. $\mathrm{dom}(\beta) = X_{[\![T]\!]}$, *and every current occurrence x has exactly one authenticated binding $\beta(x) = (j_x, \eta)$ to the domain's single current monitor version. Every current leaf b:$\mathcal{C}$ is exactly the indexed residual of its immutable registered base Γ_b by the raw projection of its resolved cursor χ_b, rather than a caller-pruned subfamily. Every constructor in T is defined, so every subtree interpretation is completion-observable.*

4) Schema and domain. *Every edge and introduced contract is justified by one immutable schema at version ζ. Every current, checkpoint, or schema occurrence, effect identity, monitor entry, lineage, and scope belongs to ω. Effect identities, receipt logs, and policies of distinct domains are disjoint, and an edit never crosses a domain.*

The installation kernel additionally requires that the policy domain's one current monitor version is active.

3) Receipt log and security state: Outside H, the security state stores policy version κ, a regular prefix-closed permission policy $\mathcal{A} \subseteq \mathcal{U}^*$ over label alphabet $\mathcal{U}$, append-only receipt log Δ, and ordered dispatch queue out. Receipt creation is append-only. Settlement advances receipt phases and fills durable return records with stable return values. Each receipt binds its effect identity, immutable normalized tool request, creator occurrence, and permission label. Each record in χ points to its own fresh receipt or, for alias, an earlier receipt, making retry and alias return lookup functions of (χ, Δ). $\mathsf{lab}(\Delta)$ is the permission trace: the ordered labels of receipt-creating calls. Restore may reconstruct workspace payload but never restores Δ.

A security core $(\kappa, H, \Delta, \mathsf{out}, \mathcal{A})$ is *well formed* when H is well formed, $\mathsf{lab}(\Delta) \in \mathcal{A}$, each effect identity has at most one fresh receipt, each fresh record names its unique receipt, each alias record names an earlier receipt, the receipt log, dispatch queue, and policy are scoped to ω, and the queue preserves fresh order. This joint invariant is assumed below.

C. Adversary and trusted boundary

The adversary controls Agent output, edit requests, tool arguments, copied workspace, retries, stale authorization tokens, and scheduling, including races between calls and edits. It cannot forge registries, the checkpoint directed acyclic graph (DAG), effect-identity certificates, policy, or receipt log. The trusted computing base (TCB) includes the policy signer and call-authorizing service. Within one policy domain, it completely mediates every protected use and provides a linearizable, crash-stable transaction for every modeled state change and visible edit answer. A recovered labeled transition system (LTS) state shows each transaction entirely uncommitted or committed according to whether its linearization point occurred. No transaction is torn, and volatile compiler caches lie outside the model. Before the checker takes a source snapshot, the call-authorizing service may instantiate finitely many pre-authorized schema templates. For tool request m, it computes normalized request $m^\circ = \mathsf{canon}_\kappa(m)$, reuses only an effect identity certified for m° or allocates a fresh one, and signs $(x, j, d, \mathsf{scope}, \mathsf{digest}(m^\circ), \mathsf{lineage})$ into the authenticated registry. The resulting finite registered call model R remains unchanged while that snapshot is checked. Unmatched calls fail closed. The platform supplies authenticated $\mathcal{W}, R$, edit schemas, normalization rules, policy, checkpoints, and one consistent snapshot of the receipt log and dispatch queue. From these inputs and an identifier-only Agent request, the controller derives the edited workflow, required outcomes, resolved calls, fixed point, monitor, certificate, monitor version, and authorization tokens. The Agent may supply or weaken none of these derived objects.

CheckReg accepts a finite registered call model only when it faithfully represents the outcome, order, effect identity, and receipt behavior of a boundary-finite typed Agent workflow. Definition 6 and theorem 1 give the exact check and prove a bijection between source executions at protected-call boundaries and registered executions. The theory neither infers natural-language contracts nor equates local commit with remote physical exactly-once execution, which additionally requires an idempotent or queryable sink.

D. Six Execution-Edit Rules

1) Edit Certificates: The six disjoint constructors of Edit_6 are exactly the typed operation signatures in table I. The set excludes Save, Root, Extend, protected use, and caller-supplied targets.

An untrusted request u supplies an edit kind, current object identifiers, and schema identifier σ, never a target contract, while Σ_ζ binds σ to that kind, exact source and checkpoint fingerprints, introduced suffix contracts, and a lineage certificate.

The edit certificate defined below makes the derivation checkable. For indexed source $\mathcal{C}$, schema-designated carried or cloned target region $\mathcal{D}$, and parent map $\mu : X_\mathcal{D} \to X_\mathcal{C}$, write $\mathcal{C} \preccurlyeq_\mu \mathcal{D}$ exactly when μ is a global bijection, some surjection $\pi : O_\mathcal{D} \twoheadrightarrow O_\mathcal{C}$ satisfies the conditions below for every target outcome v, and μ preserves effect identity and lineage.

$$x \in X_{p_v} \iff \mu(x) \in X_{p_{\pi(v)}},$$
$$x \prec_v y \iff \mu(x) \prec_{\pi(v)} \mu(y).$$

Global bijection and occurrence–outcome membership equivalence prevent splitting a source occurrence shared across outcomes into outcome-specific target occurrences.

For candidate T', let R_u index the disjoint carried or cloned target regions inside the replaced subtree. An edit certificate $\mathcal{P}_u = \{(\mathcal{C}_r, \mathcal{D}_r, \mu_r, \pi_r)\}_{r \in R_u}$ contains the relation for each indexed region. A separate identity map checks that the exposed one-hole context preserves syntax, outcome indices, occurrences, effect identities, lineage, and order. Writing X_{ctx} for its occurrences and $X_{\mathrm{suf(u)}}$ for schema-declared suffix occurrences, the checker verifies the following disjoint cover.

$$X_{[\![T']\!]} = X_{\mathrm{ctx}} \uplus \biguplus_{r \in R_u} X_{\mathcal{D}_r} \uplus X_{\mathrm{suf(u)}}.$$

The row constructor and exposed-context syntax induce $\mathsf{pr}^u_r : O_{[\![T']\!]} \rightharpoonup O_{\mathcal{D}_r}$. For choice, this projection is defined only in the arm containing r, while product and sequence select the corresponding indexed component. Define

$$\mathsf{RegionReq}_u = \{(r, o) \mid r \in R_u,\ o \in O_{\mathcal{C}_r}\},$$
$$\mathsf{Lift}_u(t, r, o) \iff \exists v \in O_{\mathcal{D}_r}.\quad \mathsf{pr}^u_r(t) = v \ \wedge\ \pi_r(v) = o.$$

Let O_{byp} be source-context outcomes whose alternatives bypass the hole, and let $\iota^u_{\mathrm{ctx}} : O_{\mathrm{byp}} \hookrightarrow O_{[\![T']\!]}$ be the injection induced by context identity. Define the single required-outcome set and its bypass lift by

$$\mathsf{Req}_u = \mathsf{RegionReq}_u \uplus (\{\mathsf{ctx}\} \times O_{\mathrm{byp}}),$$
$$\mathsf{Lift}_u(t, (\mathsf{ctx}, o)) \iff t = \iota^u_{\mathrm{ctx}}(o).$$

For regional $a = (r, o) \in \mathsf{RegionReq}_u$, $\mathsf{Lift}_u(t, a)$ denotes the relation above. The checker also requires $\forall a \in \mathsf{Req}_u.\ \exists t \in O_{[\![T']\!]}.\ \mathsf{Lift}_u(t, a)$, ensuring a full-target lift for every required source outcome, no outcome-specific splitting of shared occurrences, and no inherited occurrence mislabeled as a suffix.

The schema-verification phase checks the following judgment.

$$\Sigma_\zeta; H \vdash_{\mathrm{edit}} u \Rightarrow (T', \mathcal{P}_u)$$

It holds exactly when every named current object belongs to $\mathsf{addr}(T)$, every introduced object remains scoped to ω, the source fingerprint matches, the applicable $\mathcal{P}_u$, disjoint cover, and Lift_u requirements in table I hold, any removal is authorized, and $[\![T']\!]$ is defined. Authorized removal is confined to a typed MergeSelect's losing arm or to a RestoreReplace's current continuation that is complete or explicitly certified for removal. The rule checks these recorded facts rather than Agent declarations. The removal certificate is an immutable record signed by the policy authority at version ζ. It binds (ω, σ), the exact source fingerprint, the exact outcome indices and occurrence lineages that may be removed, and verifies that no other occurrence disappears. Its certified set must equal the typed losing arm for MergeSelect or the current continuation for RestoreReplace.

2) Preserving required outcomes: A row-tagged source outcome identifies one outcome that the edit must account for. Req_u contains those that must still be realized in the edited workflow. The occurrences of such an outcome are the calls needed to realize it.

Independently of T', let $\mathsf{SourceReq}(H, u)$ be all row-tagged source outcomes, $\mathsf{Completed}_{H,u}$ those already completed at the current cursor, and $\mathsf{Removed}_{H,u}$ the exact set whose removal the policy authority has signed. The checker enforces *required-outcome preservation*:

$$\mathsf{SourceReq}(H, u) = \mathsf{Req}_u \uplus \mathsf{Completed}_{H,u} \uplus \mathsf{Removed}_{H,u}.$$

Checked lifts preserve effect identity, lineage, and order. The installed edit retains completion evidence and records the authorized removal set, which the Agent cannot enlarge.

Let $\mathcal{E}[-]$ be a one-hole *exposed* active-workflow context, so its hole has the addressability property above. carry preserves continuation, outcome, and occurrence identifiers, pomset order, effect identities, lineage, base contracts, current branch cursors, and nested choice status. clone_r gives continuation, outcome, and occurrence identifiers deterministic fresh names while preserving pomset order and authenticated effect identity, scope, digest, and lineage data. It registers each copied remainder as a new base contract with an empty cursor. Schema suffix leaves likewise start with their registered base contract and an empty cursor. The schema-derived contracts $E^L_\sigma, E^R_\sigma, E^J_\sigma$ are suffixes, not replacements. Define

$$L_\sigma = \mathsf{clone}_L(\mathcal{C}) \triangleright E^L_\sigma, \qquad R_\sigma = \mathsf{clone}_R(\mathcal{C}) \triangleright E^R_\sigma.$$

New identifiers and the target monitor version η^+ are deterministically allocated from $(\omega, \zeta, \nu, u, \mathsf{role})$. role identifies the edit row and operand position. Write $\mathsf{Cl}_r(\mathcal{C})$ for the certified tuple $(\mathcal{C}, \mathsf{clone}_r(\mathcal{C}), \mu_r, \pi_r)$, and $\mathsf{Ca}(\mathcal{C})$ for $(\mathcal{C}, \mathsf{carry}(\mathcal{C}), \mu, \pi)$. Each abbreviation asserts the corresponding $\preccurlyeq_\mu$ relation. For an active workflow T_0, $\mathsf{Ca}(T_0)$ abbreviates $\mathsf{Ca}([\![T_0]\!])$ and has target contract $[\![\mathsf{carry}(T_0)]\!]$. Every b or g named below must lie in $\mathsf{addr}(T)$.

In an open choice, the first fresh or alias atomically selects its arm and disables later occurrences from the other, so $\mathsf{MergeSelect}(g, w, \sigma)$ has no derivation after any durable loser progress, including receipt-silent progress. MergeJoin replaces the addressable $\|_g$ node with jbar, removes g from the addressable workflow, and prevents the same pair from being joined again.

The model replaces the monitor for the whole policy domain rather than disabling only selected monitor entries, so every current entry in H belongs to $\eta^- = \eta$.

3) Replacing the Monitor Version: The execution-edit judgment is the least relation generated by one row of table I inside a well-typed single-hole context $\mathcal{E}$, followed by the common update below; no other execution-edit derivation exists. Let e_u contain its typed edge and exactly the newly allocated clone, suffix, and group nodes prescribed by T'; carried node identities remain unchanged. For every current target occurrence x, the trusted call-authorizing service deterministically mints j^+_x, h^+_x, and ρ' preserves the certified effect identity, lineage, scope, digest, and records needed for Retry while refreshing monitor entries and authorization tokens. Define

$$\mathcal{H}^+_u = \{(x, j^+_x, h^+_x) \mid x \in X_{[\![T']\!]}\},$$
$$\beta'(x) = (j^+_x, \eta^+).$$

TABLE I
SCHEMA-CHECKED AGENT EXECUTION EDITS. $E^{\bullet}_{\sigma}$ IS DERIVED FROM Σ_{ζ}, NEVER SUPPLIED AS A TARGET CONTRACT.

Operation u	Required current shape	Derived target shape	Checked preservation
$\mathsf{ForkChoice}(b,\sigma)$	$\mathcal{E}[b{:}\mathcal{C}]$	$\mathcal{E}[b_L{:}L_\sigma \oplus_g^{\mathsf{open}} b_R{:}R_\sigma]$	$\{\mathsf{CI}_L(\mathcal{C}), \mathsf{CI}_R(\mathcal{C})\}$
$\mathsf{ForkParallel}(b,\sigma)$	$\mathcal{E}[b{:}\mathcal{C}]$	$\mathcal{E}[b_L{:}L_\sigma \parallel_g b_R{:}R_\sigma]$	$\{\mathsf{CI}_L(\mathcal{C}), \mathsf{CI}_R(\mathcal{C})\}$
$\mathsf{RestoreReplace}(b,k,\sigma)$	$\mathcal{E}[b{:}\mathcal{C}],\ k \in \operatorname{dom} K$	$\mathcal{E}[b_k{:}\mathsf{clone}_k(\mathcal{C}_k)]$	$\{\mathsf{CI}_k(\mathcal{C}_k)\}$; current continuation removed
$\mathsf{RestoreLive}(b,k,\sigma)$	$\mathcal{E}[b{:}\mathcal{C}],\ k \in \operatorname{dom} K$	$\mathcal{E}[b{:}\mathsf{carry}(\mathcal{C}) \parallel_g b_k{:}\mathsf{clone}_k(\mathcal{C}_k)]$	$\{\mathsf{Ca}(\mathcal{C}), \mathsf{CI}_k(\mathcal{C}_k)\}$
$\mathsf{MergeSelect}(g,w,\sigma)$	$\mathcal{E}[T_L \oplus_g^s T_R],\ s \in \{\mathsf{open}, w\}$	$\mathcal{E}[\mathsf{carry}(T_w); b_J{:}E^J_\sigma]$	$\{\mathsf{Ca}(T_w)\}$; other arm removed
$\mathsf{MergeJoin}(g,\sigma)$	$\mathcal{E}[T_L \parallel_g T_R]$	$\mathcal{E}[\mathsf{jbar}(\mathsf{carry}(T_L), \mathsf{carry}(T_R)); b_J{:}E^J_\sigma]$	$\{\mathsf{Ca}(T_L), \mathsf{Ca}(T_R)\}$

The replacement-token bundle is inactive and inaccessible to the caller before installation. The complete judgment is shown below.

$$\begin{aligned}\Sigma_\zeta; H \vdash u \Downarrow (H', \mathcal{C}_{H,u}, \eta^-, \eta^+),\\ H' = (\omega, G \cdot e_u, T', K, \Sigma_\zeta, \rho', \beta', \chi, \nu+1, \eta^+),\\ \mathcal{C}_{H,u} = [\![T']\!], \qquad \eta^- = \eta.\end{aligned} \tag{4}$$

Here $G \cdot e_u$ retains every source progress record and every removed or carried node, and initializes each new clone or suffix node with its residual base and no progress records. Thus an edit preserves K, Σ_ζ, χ, appends one typed provenance edge and its target nodes, refreshes every current monitor entry and authorization token, and moves the domain to η^+. Table constructors determine new group modes, while carried nested modes remain unchanged. A request naming a nonexistent checkpoint, wrong group mode, nonmember winner, mismatched schema fingerprint, unauthorized removal, or undefined composition has no derivation. Independent domains step by asynchronous product, and no theorem allows disabling only a selected subset of one domain's monitor entries.

Lemma 2 (Schema fidelity and complete monitor replacement)**.** *For well-formed H, the judgment in equation* (4) *is deterministic. If it derives H', then H' is well formed. It preserves required outcomes: completion evidence remains recorded, and no pending source lineage disappears outside the exact set* $\mathsf{Removed}_{H,u}$*. The identity-preserved context and family $\mathcal{P}_u$ preserve global occurrence sharing, effect identity, lineage, and causal order. For every $a \in \mathsf{Req}_u$, some full target outcome t satisfies* $\mathsf{Lift}_u(t,a)$*, including the identity lift of every bypassing context outcome. Every current target occurrence has a monitor entry bound to η^+ and one authorization token in $\mathcal{H}_u^+$, and none remains bound to η^-.*

Proof. Unique object and schema identifiers select one row and record, while context identity preserves everything outside the hole. The Fork rows certify both clones; RestoreReplace certifies the checkpoint clone and authorized removal of the source continuation, while RestoreLive certifies the carried source and checkpoint clone. MergeSelect certifies the winner and typed loser, while MergeJoin certifies both carried regions before its barrier. Clone and carry supply the global bijections and occurrence–outcome membership preservation; constructor induction defines pr_r^u, proves the cover, and supplies every Lift_u witness. The common poststate fixes all remaining fields and preserves or initializes branch cursors as specified above, while new identifiers, partial-constructor checks, and lemma 1 preserve uniqueness, acyclicity, and contract well-formedness. Finally, β' and $\mathcal{H}_u^+$ bind every current target occurrence to η^+. □

E. Registered extension

A registered extension adds new schemas and policy entries without changing the current workflow or its required outcomes. It is not a seventh execution edit. The Agent may name only an immutable record ξ binding the exact predecessor schema, registry, and policy roots to finite disjoint additions D_Σ, D_ρ and a successor policy $\mathcal{A}^+$. The policy signer and call-authorizing service both authenticate ξ. Existing schema identifiers, occurrence–effect-identity and invocation rows, labels, scopes, lineages, receipts, and normalization rules retain their meanings. New identifiers are fresh, finite, domain-scoped, foreign-key closed, and completion-observable. The record contains no cursor, receipt, dispatch-queue entry, monitor version, return value, monitor, or installed certificate. The successor policy is regular and prefix closed and conservatively extends the old policy, $\mathcal{A} \subseteq \mathcal{A}^+$.

For fresh η^+, refresh every current monitor entry and authorization token but leave the active workflow unchanged:

$$\begin{aligned}H_\xi^+ = (\omega, G, T, K, \Sigma_{\zeta^+}, \rho^+, \beta^+, \chi, \nu+1, \eta^+),\\ \mathcal{C}_\xi = [\![T]\!], \qquad \mathsf{SourceReq}(H,\xi) = \mathsf{Req}_\xi = O_{[\![T]\!]},\\ \mathsf{Completed}_{H,\xi} = \mathsf{Removed}_{H,\xi} = \varnothing, \qquad \mathsf{Lift}_\xi(t,o) \iff t = o.\end{aligned}$$

The checked judgment

$$(\kappa, H, \mathcal{A}) \vdash \mathsf{Extend}(\xi) \Downarrow (\kappa^+, H_\xi^+, \mathcal{A}^+, \mathcal{C}_\xi, \eta, \eta^+)$$

preserves $G, T, K, \chi, \Delta, \mathsf{out}$, every current outcome, occurrence, effect identity, lineage, and causal edge, and authorizes no removal. It appends one authenticated version record and then uses the same checker and atomic monitor installation as an execution edit.

IV. EXACT SAFETY CHECK FOR EXECUTION EDITS

The preceding section derives an edited workflow from the authenticated source record and requested edit. This section decides whether that workflow has a policy-safe implementation that preserves every still-possible required outcome. It first determines whether each copied call needs a new receipt or reuses an existing one. It then computes the largest family

of completed executions that remains safe after every partial execution. Finally, it presents the ideal atomic execution machine used to prove the installed runtime correct in section V.

Recall the permission-label alphabet $\mathcal{U}$ and regular prefix-closed policy $\mathcal{A} \subseteq \mathcal{U}^*$. For the authenticated receipt log Δ, $\mathsf{lab}(\Delta) \in \mathcal{A}$ is its permission trace and D_Δ is the set of effect identities that already have receipts. Fix $\Theta = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, u)$ and write $\mathsf{src}(\Theta) = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A})$. When defined, the tagged root, execution-edit, or registered-extension derivation is unique. Write $\mathsf{Derive}(\Theta) = (\kappa_u, H_u, \mathcal{A}_u, \mathcal{C}_u, \eta^-, \eta^+)$. Root derivation additionally requires that $\mathsf{CheckReg}(\mathcal{W}, R)$ accept. An execution edit has $(\kappa_u, \mathcal{A}_u) = (\kappa, \mathcal{A})$ and uses the unique execution-edit judgment. $\mathsf{Extend}(\xi)$ uses the authenticated judgment in section III. Let ρ_u be the authenticated target registry in H_u, let $O_{H,u} = O_{\mathcal{C}_u}$, and use $H' = H_u$ and $\rho' = \rho_u$ where earlier notation is convenient. All languages below depend on Θ, the schema registry, ρ_u, and target policy $\mathcal{A}_u$, but not on the dispatch queue out. The executable checker nevertheless binds out into the authenticated source snapshot used for installation. We write only H, u where the remaining parameters are fixed.

A. fresh *and* alias *resolution*

Resolution never erases an occurrence retained by the derived contract. Starting with $D_0 = D_\Delta$, the deterministic transducer $\mathsf{Resolve}_\Delta^{\rho_u}$ maps a raw word $w = x_1 \cdots x_n$ to a resolved word $z_1 \cdots z_n$. For $d_i = q_{\mathrm{cell}}^{\rho_u}(x_i)$,

$$z_i = \begin{cases} \mathsf{fresh}(x_i, d_i), & d_i \notin D_{i-1}, \quad D_i = D_{i-1} \cup \{d_i\}; \\ \mathsf{alias}(x_i, d_i), & d_i \in D_{i-1}, \quad D_i = D_{i-1}. \end{cases} \tag{5}$$

The permission projection erases alias symbols and maps each fresh symbol to its effect identity's permission label.

$$\mathsf{auth}(\mathsf{fresh}(x, d)) = \ell_{\rho_u}(d), \qquad \mathsf{auth}(\mathsf{alias}(x, d)) = \epsilon.$$

For resolved z, $\mathsf{raw}(z)$ drops the fresh/alias mode and effect identity but preserves the ordered occurrence identifiers. A Retry repeats the same invocation and adds no occurrence. Restored occurrences sharing an effect identity remain distinct and may resolve differently.

Lemma 3 (Resolution)**.** $\mathsf{Resolve}_\Delta^{\rho_u}$ *is deterministic, length preserving, and prefix compatible. Its* fresh *projection contains each effect identity at most once outside D_Δ, while its occurrence projection is exactly the input word. Moreover, if $\mathsf{Resolve}_\Delta^{\rho_u}(rv) = zv'$, then $z = \mathsf{Resolve}_\Delta^{\rho_u}(r)$ and $v' = \mathsf{Resolve}_{\Delta_z}^{\rho_u}(v)$, where Δ_z extends Δ by the* fresh *events of z.*

Proof. Induction on input length shows that the transducer emits one prefix-determined output symbol per input symbol and updates D_i only for fresh. Set membership ensures one fresh emission per effect identity. Both cases retain x_i. Splitting after r gives the streaming equation. □

Thus a retry changes neither the resolved call record nor the permission trace, although its successful API reply is observable as $\mathsf{Return}(t, [v]_\cong)$. A copied occurrence that resolves as alias advances the call record without changing the permission trace.

B. Safe Completion After Every Prefix

1) Largest family safe after every prefix: For each derived outcome $o \in O_{H,u}$, define all of its completed executions and the subset allowed by policy.

$$\begin{aligned} R_o(H, u) &= \{\mathsf{Resolve}_\Delta^{\rho_u}(w) \mid w \in \mathsf{Lin}(p_o)\}, \\ M_o(H, u) &= \{z \in R_o(H, u) \mid \mathsf{lab}(\Delta)\mathsf{auth}(z) \in \mathcal{A}_u\}. \end{aligned} \tag{6}$$

Checking each outcome separately is insufficient. After a partial execution shared by several outcomes, every outcome still compatible with that partial execution must retain a policy-safe completion. The operator below removes exactly the completed executions that violate this condition. For a resolved prefix z, let

$$\mathsf{Compat}(z) = \{o \mid \mathsf{raw}(z) \preceq_{\mathrm{pref}} w \text{ for some } w \in \mathsf{Lin}(p_o)\}.$$

Put

$$\widehat{B}_0 = \biguplus_{o \in O_{H,u}} \{o\} \times M_o.$$

For $\widehat{B} \subseteq \widehat{B}_0$, define the monotone operator

$$\widehat{\Phi}(\widehat{B}) = \left\{ (o, b) \in \widehat{B}_0 \;\middle|\; \begin{array}{l} \forall z \preceq_{\mathrm{pref}} b.\ \forall v \in \mathsf{Compat}(z). \\ \exists (v, b') \in \widehat{B}.\ z \preceq_{\mathrm{pref}} b' \end{array} \right\}. \tag{7}$$

Set $\widehat{B}_{i+1} = \widehat{\Phi}(\widehat{B}_i)$. Since $\widehat{B}_1 \subseteq \widehat{B}_0$, monotonicity yields a finite descending chain, whose limit is

$$\begin{aligned} &\widehat{B}^\dagger_{H,u} = \nu \widehat{B}.\widehat{\Phi}(\widehat{B}) = \bigcap_{i \geq 0} \widehat{B}_i, \\ &B^\dagger_{H,u} = \pi_2 \widehat{B}^\dagger_{H,u}, \qquad\qquad W^\dagger_{H,u} = \mathsf{Pref}(B^\dagger_{H,u}). \end{aligned}$$

Runtime uses $(W^\dagger, B^\dagger)$, while the checker and seal retain indexed witness $\widehat{B}^\dagger$.

Definition 2 (Safe execution edit)**.** *If* $\mathsf{Derive}(\Theta)$ *is undefined, then* $\mathsf{SafeEdit}(\Theta)$ *is false. Otherwise,*

$$\mathsf{SafeEdit}(\Theta) \iff \widehat{B}^\dagger_{H,u} \neq \varnothing. \tag{8}$$

$\mathsf{SafeEdit}(H, u)$ fixes Θ and both registries. At $z = \epsilon$, nonemptiness covers every required outcome. The existential quantifier chooses a call order without assuming fairness, and the universal condition keeps every compatible outcome completable after every prefix. Relation Preserve requires every source outcome that is neither completed nor authorized for removal to have a checked lift into $\widehat{B}$; definition 5 gives its full definition.

Definition 3 (Indexed realization)**.** *A generated language and indexed marked family $(L, \widehat{B})$ realize (H, u) when all conditions below hold.*

1) $\widehat{B} \neq \varnothing$;
2) source and target fidelity: $\mathsf{Preserve}(H, u, \widehat{B})$, $\widehat{B} \subseteq \biguplus_{o \in O_{H,u}} \{o\} \times R_o(H, u)$, *and* $L \subseteq \mathsf{Pref}(\bigcup_{o \in O_{H,u}} R_o(H, u))$;
3) policy safety: $\mathsf{lab}(\Delta)\,\mathsf{auth}(z) \in \mathcal{A}_u$ *for every* $z \in L$;
4) completion nonblocking: $L = \mathsf{Pref}(\pi_2 \widehat{B})$; *and*

5) persistent outcome coverage: *for every* $z \in L$ *and* $o \in \mathsf{Compat}(z)$*, some* $(o, b) \in \widehat{B}$ *extends* z*.*

Lemma 4 (Largest safe realization)**.** *A realization exists iff* $\mathsf{SafeEdit}(H, u)$*. When the edit is safe,* $(W^\dagger_{H,u}, \widehat{B}^\dagger_{H,u})$ *is a realization, and every realization* $(L, \widehat{B})$ *satisfies* $\widehat{B} \subseteq \widehat{B}^\dagger_{H,u}$ *and* $L \subseteq W^\dagger_{H,u}$*.*

Proof. Fidelity, completion nonblocking, and safety place any realization's indexed marked family inside $\widehat{B}_0$. Persistent coverage makes it post-fixed, so monotonicity and finite Knaster–Tarski place it inside $\widehat{B}^\dagger$, and projected prefixes give $L \subseteq W^\dagger$. Conversely, $\widehat{B}^\dagger = \widehat{\Phi}(\widehat{B}^\dagger)$ supplies persistent coverage. Together with checked lifts, this coverage establishes $\mathsf{Preserve}$ at ϵ. The inclusion $\widehat{B}^\dagger \subseteq \widehat{B}_0$ supplies target fidelity and safe marked words, while prefix closure of $\mathcal{A}_u$ supplies prefix safety. Finally, $W^\dagger = \mathsf{Pref}(\pi_2 \widehat{B}^\dagger)$ supplies nonblocking. Hence the fixed point is a realization exactly when nonempty. □

$\mathsf{GReal}(\Theta; L, \widehat{B})$ declaratively denotes a realization containing every other one componentwise, and its unique witness is $\mathsf{Spec}(\Theta)$. Lemma 4 proves that this witness exists exactly when $\widehat{B}^\dagger_{H,u} \neq \varnothing$ and then equals $(W^\dagger_{H,u}, \widehat{B}^\dagger_{H,u})$.

The following shared-prefix example shows why each outcome needs its own completion condition even when every M_o is nonempty. Let singleton contracts X, Y, Z contain x_o, y_o, z_o, and let the target registry map them to distinct effect identities d_x, d_y, d_z that do not yet have receipts and have labels x, y, z, respectively. Write $\bar{x} = \mathsf{fresh}(x_o, d_x)$, $\bar{y} = \mathsf{fresh}(y_o, d_y)$, and $\bar{z} = \mathsf{fresh}(z_o, d_z)$.

2) Why Individual Outcome Safety Is Insufficient: For $\mathcal{C} = X \otimes (Y \oplus Z)$ and $\mathcal{A} = \mathsf{Pref}\{yx, xz\}$, the $X \otimes Y$ and $X \otimes Z$ outcomes initially have safe completed executions $\bar{y}\bar{x}$ and $\bar{x}\bar{z}$, with permission traces yx and xz. Yet prefix $\bar{x}$ from $\bar{x}\bar{z}$ remains contract-compatible with $X \otimes Y$ but leaves no safe continuation. The first fixed-point iteration removes $\bar{x}\bar{z}$. The next removes $\bar{y}\bar{x}$ because the empty prefix no longer has a completion for every outcome. Thus the edit is rejected, whereas the independent $\forall o.\, M_o \neq \varnothing$ test accepts it.

Proposition 1 (Outcome-indexed nonblocking correspondence)**.** *For nonempty* $\widehat{B} \subseteq \widehat{B}_0$*, let* $G_{\widehat{B}}$ *be the prefix trie of* $L_{\widehat{B}} = \mathsf{Pref}(\pi_2 \widehat{B})$*. Mark state* z *by* α_o *exactly when* $o \in \mathsf{Compat}(z)$*, and mark state* b *by* ω_o *exactly when* $(o, b) \in \widehat{B}$*. Then*

$$\widehat{B} \subseteq \widehat{\Phi}(\widehat{B}) \iff \forall o \in O_{H,u}.\, G_{\widehat{B}} \text{ is } (\alpha_o, \omega_o)\text{-nonblocking}.$$

Consequently, if $\widehat{B}^\dagger$ *is nonempty, it is the largest nonempty completion subfamily of* $\widehat{B}_0$ *satisfying these outcome-specific nonblocking conditions. If it is empty, no nonempty subfamily satisfies the conditions. Ordinary marker nonblocking of the unaugmented projected language is strictly weaker.*

Proof. From reachable prefix z, an ω_o-marked state is reachable exactly when some $(o, b) \in \widehat{B}$ extends z. Quantifying over exactly the states marked α_o is therefore the post-fixed-point condition. Greatestness follows from the greatest-fixed-point construction. For strictness, the preceding instance has the marker-nonblocking projected pair $(\mathsf{Pref}\{\bar{y}\bar{x}, \bar{x}\bar{z}\}, \{\bar{y}\bar{x}, \bar{x}\bar{z}\})$, but its indexed fixed point is empty. □

Proposition 1 gives the exact interface to generalized nonblocking [15], [16]. After the trusted controller derives α_o, ω_o from the requested edit, authorized removals, effect identities, receipts, and source snapshot, a generalized-nonblocking solver computes the same largest family. Complete mediation makes fresh/alias controllable. Deployments with uncontrollable resolved events need the usual supremal-controllable refinement. Our theory additionally constructs the most permissive Agent monitor or an impossibility certificate, carries the result through atomic installation, and proves that neither a caller-proposed workflow nor its traces contain enough information to reproduce the derivation.

Each removed (o_{rem}, b) records its rank i, a prefix $z \preceq b$, and uncovered $o_{\mathrm{miss}} \in \mathsf{Compat}(z)$. Induction shows that every post-fixed P is contained in every $\widehat{B}_i$. An empty final family therefore excludes every realization, and installation recomputes the ranked rejection certificate and the remaining witnesses.

3) Rejection certificates and repeated checking: To show that the checker can be applied again after each permitted partial execution, fix $H, u, \mathcal{A}_u$ after deriving $\mathcal{C}$, and write $W^\dagger(\mathcal{C}, \Delta)$ and $\widehat{B}^\dagger(\mathcal{C}, \Delta)$ with those parameters suppressed.

Lemma 5 (Residual coherence)**.** *If* $z \in W^\dagger(\mathcal{C}, \Delta)$*, then* $\mathsf{Resolve}^{\rho'}_{\Delta}(\mathsf{raw}(z)) = z$*,* $\mathsf{lab}(\Delta_z) = \mathsf{lab}(\Delta)\mathsf{auth}(z)$*, and*

$$W^\dagger(\mathcal{C}, \Delta)/z = W^\dagger(\mathcal{C}/\mathsf{raw}(z), \Delta_z),$$
$$\widehat{B}^\dagger(\mathcal{C}, \Delta)/z = \widehat{B}^\dagger(\mathcal{C}/\mathsf{raw}(z), \Delta_z),$$

where $\widehat{B}/z = \{(o, v) \mid (o, zv) \in \widehat{B}\}$*.*

Proof. Because z prefixes a projected member of $\widehat{B}^\dagger$, raw recoverability and streaming in lemma 3 give the recoverability and receipt-log equalities. The indexed residual removes exactly that raw prefix, retains every compatible outcome identity, and gives

$$\mathsf{Compat}_{\mathcal{C}/\mathsf{raw}(z)}(s) = \mathsf{Compat}_{\mathcal{C}}(zs).$$

Together with authority concatenation, streaming resolution therefore yields the key equality

$$\widehat{B}_0(\mathcal{C}, \Delta)/z = \widehat{B}_0(\mathcal{C}/\mathsf{raw}(z), \Delta_z),$$

with the same retained outcome identities, policy, and target registry. Thus $\widehat{B}^\dagger/z$ is post-fixed for the residual operator and lies within its greatest fixed point Y. Conversely, streaming and residual policy safety put $zY = \{(o, zy) \mid (o, y) \in Y\}$ inside the original $\widehat{B}_0$. The union $\widehat{B}^\dagger \cup zY$ is post-fixed because prefixes $t \preceq z$ use existing witnesses, while prefixes $t = zs$ use witnesses in Y. Greatestness gives $zY \subseteq \widehat{B}^\dagger$, hence $Y = \widehat{B}^\dagger/z$. Finally, $\mathsf{Pref}(B)/z = \mathsf{Pref}(B/z)$ yields the generated-language equality. □

Thus every accepted prefix retains a safe completion for every residual outcome index still contract-compatible with that prefix.

4) Registered extension:

Lemma 6 (Registered extension preserves safety)**.** *For an authenticated* $\mathsf{Extend}(\xi)$*, the current residual indexed realization is a post-fixed family for the extension instance and is therefore contained in* $\widehat{B}^{\dagger}_{H,\mathsf{Extend}(\xi)} \neq \varnothing$*. Consequently, a valid registered extension cannot remove a currently required outcome or leave it without a safe completion.*

The proof is in section A-B.

C. The ideal atomic execution machine

An ideal state $I = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, c, r)$, with $c = (H_c, u, L^*, \widehat{B}^*)$, keeps only the authenticated execution data, current monitor version and entries, installed specification, and resolved prefix. It omits compilation, monitor representation, transaction phases, authenticators, preloading, and installation microsteps.

a) Edit transition: For current $\Theta_I(u)$, an undefined derivation gives state-preserving Invalid, while a defined derivation with no GReal witness gives state-preserving Reject. Otherwise $\mathsf{Derive}(\Theta_I(u))$ supplies the target and monitor version, and independently defined $\mathsf{Spec}(\Theta_I(u)) = (L^*, \widehat{B}^*)$ enables one atomic $\mathsf{Edge}(u, [\mathcal{H}^+_u]_{\cong})$ that installs them and resets r to ϵ. All three answers bind the same current source-snapshot token and are mutually exclusive and exhaustive.

b) Protected call: For submitted tool request m, an enabled x passes the ideal guard exactly when its current entry and authorization token bind effect identity d, $\mathsf{canon}_{\kappa}(m) = \mathsf{inv}_{\rho}(d)$, and $ra \in L^*$, where receipts choose $a = \mathsf{fresh}(x, d)$ or $\mathsf{alias}(x, d)$. One atomic transition applies HStep, appends a to χ, advances ν, removes x's binding, and sets $r := ra$; fresh also creates the labeled receipt and queues the normalized request, whereas alias links to the existing return record. Any failed premise returns $\mathsf{deny}(x)$ without changing state or inventing a return value.

c) Retry, release, and crash: Save, authenticated retry, Dispatch, settlement, and recovered crash have the same durable behavior as their compiled rules in table III; compiler computation is absent. This least-generated LTS makes no claim about external-network completion order after Dispatch.

By lemma 4, every reachable partial execution of the ideal machine retains a policy-safe completion until the next edit transition changes the workflow contract. The next sections implement this machine without consulting the full completion set at every call.

V. Building and Installing the Runtime Monitor

The safety checker produces either an impossibility certificate or the largest safe family of completed executions. The installation kernel turns a nonempty family into runtime enforcement in three steps. It builds a monitor for the family's partial executions, binds that monitor to the authenticated source snapshot, and atomically installs it under a new monitor version. The rules below serialize every overlapping protected call either before or after installation.

A. Building an occurrence-sensitive monitor

The greatest realization has the finite, canonically installable projected pair $(W, B) = (W^{\dagger}_{H,u}, B^{\dagger}_{H,u})$. For $L/z = \{v \mid zv \in L\}$, use residual states $q_z = (W^{\dagger}/z, B^{\dagger}/z)$.

$$\begin{aligned} Q_{W,B} &= \{q_z \mid z \in W\}, \\ q^0 &= q_{\epsilon} = (W^{\dagger}, B^{\dagger}), \\ q_z \xrightarrow{a} q_{za} \quad &\Longleftrightarrow \quad za \in W. \end{aligned} \tag{9}$$

Call this monitor automaton $\mathsf{Can}(W, B)$. Every state is reachable and is marked exactly when $\epsilon \in B^{\dagger}/z$. Each transition symbol a contains occurrence x, effect identity d, and its fresh or alias mode, so one call position cannot use another position's monitor entry.

Lemma 7 (Exact compiled realization)**.** *The generated and marked languages of the program in equation* (9) *are exactly* $(W^{\dagger}_{H,u}, B^{\dagger}_{H,u})$*. The program is the smallest deterministic marked partial automaton for this pair up to state renaming.*

Proof. Induction on z shows that the automaton reaches $(W^{\dagger}/z, B^{\dagger}/z)$, enables a iff $za \in W^{\dagger}$, and marks the reached state iff $z \in B^{\dagger}$. Distinct states differ on a generated or marked suffix and must remain distinct. The automaton is the paired-language derivative construction [19]. □

For $\Theta = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, u)$, the checker returns exactly one of

$$\begin{aligned} \mathsf{CheckOutput}(\Theta) ::= \mathsf{Invalid}([\gamma, C_{\mathrm{str}}]_{\cong}) \mid \mathsf{Reject}([\gamma, C_{\mathrm{fp}}]_{\cong}) \\ \mid \mathsf{Installable}(\delta, W^{\dagger}, \widehat{B}^{\dagger}, Z). \end{aligned}$$

Invalid carries a verified undefined-derivation transcript, Reject carries the complete ranked elimination certificate, and Installable carries the unique derivation, coverage witnesses, canonical monitor, and

$$Z = (\omega, \zeta, \kappa_Z, \mathsf{cut}_Z, u, H_{\mathsf{prog}}, \eta^-, \eta^+),$$

Z authenticates the complete source snapshot, the derived target, the monitor, and both old and new monitor versions. Section A-B gives the exact payloads and proves that all three finite outputs are unique and equivariant. The bounded implementation may instead return $\mathsf{ResourceLimit}(b, n)$, which installs nothing and proves neither safety nor impossibility.

B. Installation kernel

The compiled state for one policy domain is

$$S = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, \mathsf{phase}, \mathsf{bind}, \mathsf{prog}, \mathsf{q}, \mathsf{cert}),$$

where phase records whether each monitor version η is unallocated ($\perp$), inactive, active, or closed. The bootstrap rule runs only after $\mathsf{CheckReg}(\mathcal{W}, R)$ accepts, and its exact construction appears in section A-E. Each installed version stores $c = (\mathsf{src}_c, H^+_c, u, W, \widehat{B}, Z)$: the authenticated source snapshot, derived target, edit, safe language, indexed family, and checked certificate.

Definition 4 (Agent security-state invariant)**.** *For* $c = (\mathsf{src}_c, H_c^+, u, W, \widehat{B}, Z)$ *and resolved* r, $\mathsf{AgentSec}(S; c, r)$ *holds exactly when the following clauses hold.*

1) Core and schema coherence. *The state is well formed,* $c = \mathsf{cert}(\eta)$*, its origin derives the installed target, and every* $a \in \mathsf{Req}_u$ *has a checked* Lift_u *witness.*
2) Required-outcome coherence. $\widehat{B} = \widehat{B}^{\dagger}_{H_c,u} \neq \varnothing$*,* $W = \mathsf{Pref}(\pi_2 \widehat{B})$*, and* Z *verifies both at* src_c*.*
3) Execution coherence. $(H.G, H.T) = \mathsf{HRes}((H_c^+.G, H_c^+.T), r)$*,* $H.\chi = H_c^+.\chi r$*, and receipts, queue entries, return links, and checkpoints match* r *and advance only monotonically.*
4) Monitor coherence. $\mathsf{prog}(\eta) \cong \mathsf{Can}(W, \pi_2 \widehat{B})$*, and* $\mathsf{q}(\eta)$ *is its derivative after* r*.*
5) Monitor-version coherence. *Only* η *is active; it owns exactly one authenticated entry and token for each current occurrence, while inactive or closed versions expose none.*

The full invariant and every preserving transition are enumerated in table III.

a) Atomic protected call: For submitted tool request m, set $m^\circ = \mathsf{canon}_\kappa(m)$. For x bound to monitor entry j in version η, $\mathsf{VerifyHandle}(h, \rho(x), j, d, m^\circ)$ verifies the call-authorizing service, scope, monitor entry, effect identity, lineage, and signed digest and requires $m^\circ = \mathsf{inv}_\rho(d)$. Set $a = \mathsf{fresh}(x, d)$ if $d \notin D_\Delta$, otherwise $a = \mathsf{alias}(x, d)$, and let $\mathsf{Use}_x(m) = \mathsf{Use}(x, j, h, m)$. Its sole durable transition is the rule below.

$$\frac{\begin{array}{c} x \text{ enabled in } T \quad \mathsf{bind}(x) = (j, \eta) \\ \eta = H.\mathsf{epoch} \quad \mathsf{phase}(\eta) = \mathsf{active} \\ \mathsf{VerifyHandle}(h, \rho(x), j, d, m^\circ) \quad \mathsf{q}(\eta) \xrightarrow{a}_{\mathsf{prog}(\eta)} q' \end{array}}{S \xrightarrow{\mathsf{Use}_x(m)/a} S'}. \tag{10}$$

The paired update $\mathsf{HStep}((G, T), a)$ removes x, records any choice, preserves the other workflow constructors, and appends a to its branch cursor. $\mathsf{Advance}(S, x, a, q')$ changes exactly the fields below.

$$\begin{aligned} (H.G, H.T) &:= \mathsf{HStep}((G, T), a), & H.\chi &:= \chi a, \\ H.\nu &:= \nu + 1, & H.\beta &:= \beta \setminus \{x\}, \\ \mathsf{bind} &:= \mathsf{bind} \setminus \{x\}, & \mathsf{q}(\eta) &:= q'. \end{aligned}$$

In addition, fresh appends the immutable labeled receipt and ordered dispatch-queue item $(d, \mathsf{inv}_\rho(d))$, never an unchecked caller buffer. alias records the effect identity's durable return link without changing Δ, and all other fields remain unchanged.

An authenticated Retry follows the archived χ-to-receipt link for the same normalized request and returns any stable value without changing state. For unresolved x, equation (10) applies exactly when all premises hold, while a missing transition, invalid authorization token, unbound occurrence, disabled branch, or closed monitor version returns $\mathsf{deny}(x)$ without changing state. Dispatch durably releases the oldest queued request, settlement fills its return record, and neither implies exactly-once execution by a remote service.

1) Answer verification and atomic installation: At its linearization point, the kernel ignores any caller-supplied replacement workflow or monitor version, sets $\Theta_S(u) = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, u)$, and re-runs $\mathsf{Derive}(\Theta_S(u))$ from the current state. It returns state-preserving Invalid exactly for a verified undefined derivation and Reject exactly for a verified empty fixed point. For the positive case, write $\mathsf{Derive}(\Theta_S(u)) = (\kappa_u, H_u, \mathcal{A}_u, \mathcal{C}_u, \eta^-, \eta^+)$. $\mathsf{ReCut}(S, u, Z)$ is undefined without this root, execution-edit, or registered-extension derivation and otherwise recomputes the right-hand side of equation (12). Verify_Z recomputes the entire fixed point, its witnesses, (W, B), the canonical monitor, and both seals. Installation requires

$$\begin{aligned} \kappa = \kappa_Z, \qquad & \mathsf{ReCut}(S, u, Z) = \mathsf{cut}_Z, \\ & \mathsf{Verify}_Z(W^{\dagger}_{H,u}, B^{\dagger}_{H,u}, \widehat{B}^{\dagger}_{H,u}), \\ H.\mathsf{epoch} = \eta^-, \qquad & \mathsf{phase}(\eta^+) \in \{\bot, \mathsf{inactive}\}. \end{aligned} \tag{11}$$

$\mathsf{Ready}(S, u, Y)$ holds when $Y = \mathsf{Installable}(\delta, W^{\dagger}, \widehat{B}^{\dagger}, Z)$, the current derivation is δ, and all state-dependent premises of equation (11) hold in S. If these premises hold, one domain transaction verifies every target entry and authorization token. Let $c^+ = (\mathsf{src}(\Theta_S(u)), H_u, u, W^{\dagger}, \widehat{B}^{\dagger}, Z)$. It then applies

$$\begin{aligned} \kappa &:= \kappa_u, & \mathcal{A} &:= \mathcal{A}_u, \\ H &:= H_u, & \mathsf{phase}(\eta^-) &:= \mathsf{closed}, \\ \mathsf{phase}(\eta^+) &:= \mathsf{active}, & \mathsf{bind}|_{X_{[\![H_u.T]\!]}} &:= H_u.\beta, \\ \mathsf{prog}(\eta^+) &:= \mathsf{prog}_Z, & \mathsf{q}(\eta^+) &:= q^0, \\ \mathsf{cert}(\eta^+) &:= c^+. \end{aligned}$$

Only after activation does the transaction expose $\mathcal{H}_u^+$ and emit $\mathsf{Edge}(u, [\mathcal{H}_u^+]_{\cong})$; old tokens then fail, while Retry still follows its archived receipt link. A stale candidate emits no answer and must be recomputed from the current source snapshot.

2) Other transitions: Preload, Save, retry, token retrieval, return delivery, Dispatch, settlement, stale candidates, and crash have exactly the state changes listed in table III; pure Compile is outside the LTS.

Lemma 8 (Edit–call serialization)**.** *In every reachable well-formed compiled state, a visible edit answer and a protected call in the same policy domain have a unique order at the installation point. If the use commits first, that seal cannot install. If* Install *commits first, the old use cannot resolve* fresh *or* alias*. Steps in independent domains commute by product semantics.*

Proof. Prior fresh/alias invalidates every old source-snapshot seal, while prior installation closes the old monitor version. Disjoint-domain steps commute, and the persisted target and certificate restore the invariant. □

C. Decidability and trust

Proposition 2 proves an output-sensitive $O(D + n\Lambda + |\mathcal{G}_{\mathrm{cov}}| + V_B \log(2 + V_B))$-time, $O(D + \Lambda + |\mathcal{G}_{\mathrm{cov}}|)$-space bound. Exponential growth occurs only in explicitly emitted linearizations and prefix–required-outcome support pairs. The TCB consists of the registries, receipt log, atomic rules,

policy signer, call-authorizing service, certificate verifier, and complete mediation. Search, minimization, and preloading remain untrusted because installation checks them again.

VI. The Exact Safety Theorem

The main theorem states the paper's complete theory in one place. It connects authenticated edit derivation, the safety checker, monitor construction, and atomic installation. Here *exact* means that the checker accepts if and only if a safe implementation exists. Clause 2 is the central equivalence. Clauses 1, 3, 4, and 5 prove faithful registration, the necessity of all four state components, repeated applicability, and correctness of the installed runtime.

Fix a derivation witness $\delta = (\kappa_u, H_u, \mathcal{A}_u, \mathcal{C}_u, \eta^-, \eta^+)$. An independently defined *installed realization* is a pair $(M, \widehat{B}_M)$ such that $(\mathsf{Gen}(M, q^0), \widehat{B}_M)$ realizes (H, u) according to definition 3 and $\mathsf{Mark}(M, q^0) = \pi_2 \widehat{B}_M$. The domain transaction must atomically install $(\kappa_u, H_u, \mathcal{A}_u, M)$ at η^+, close η^-, bind the target monitor entries, and publish replacement authorization tokens.

a) Information required by any exact checker: Let S be the compiled runtime state and AgentSec its security invariant. At a checked source snapshot, $\mathsf{Sec}(S; c, r) = \mathbf{P} \bowtie \mathbf{I} \bowtie \mathbf{E} \bowtie \mathbf{C}$ joins four kinds of information: workflow structure and edit schemas ($\mathbf{P}$), call-to-effect identity ($\mathbf{I}$), completed and pending effects ($\mathbf{E}$), and monitor-version and installation coordinates ($\mathbf{C}$). States are compared up to consistent renaming of private identifiers. $\mathcal{Q}_{\mathrm{sec}}$ contains every well-typed decide or install query. For an information map f, define

$$\mathsf{Exact}(f) \iff \exists D_f.\ \forall (V, q) \in \mathcal{Q}_{\mathrm{sec}},\ D_f(\langle f(V), q \rangle_{\cong}) = [\mathsf{Ans}(V, q)]_{\cong}.$$

Thus f is exact when some decoder can reproduce every correct answer from the retained information. For $J \subseteq \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}$, projection π_J keeps the factors in J and removes every field that depends on a discarded factor. TargetOnly keeps only the replacement workflow and completion data proposed by the caller. Section A-D gives the complete field dependencies and query semantics. The lower bound concerns information, not its storage layout.

b) Required-outcome preservation: The main theorem uses the following compact condition from definition 5:

$$\mathsf{Preserve}(H, u, \widehat{B}) \iff \forall a \in \mathsf{Req}_u.\ \exists t \in O_{H,u}, b,\ \mathsf{Lift}_u(t, a) \wedge (t, b) \in \widehat{B}.$$

It says that every required source outcome appears in some accepted target outcome.

c) Visible runtime behavior: $\mathcal{O}_{\mathrm{api}}$ exposes edit answers bound to a specific source snapshot, installed edits, protected-call answers, dispatch, settlement, and authenticated returns. It quotients private names but preserves public bytes. $\mathsf{obs}_{\mathrm{api}}$ hides only internal computation, preload, Save, stale attempts, and recovered crash. $\mathcal{R}$ relates ideal and compiled states when AgentSec, source snapshot and current partial execution, monitor version, current bindings, and visible fields agree. Section A-H gives its typed maps, reply linkage, and good initial pairs.

Theorem 1 (Exact Edit-Safety)**.** *Clause 1 holds for every finite-state $\mathcal{W}$ and finite R. For Clauses 2–5, assume that registration has been accepted, every resolved protected use is completely mediated and controllable, and every modeled state change within one policy domain uses a crash-stable linearizable transaction. Then the clauses hold for every $u \in \mathsf{Edit}_6 \uplus \{\mathsf{Extend}(\xi)\}$ at every finite reachable state satisfying* AgentSec*. Registration acceptance entails the boundary-finiteness required by the checker.*

1) ***Faithful registration.*** $\mathsf{CheckReg}(\mathcal{W}, R)$ *accepts iff stored λ witnesses $\mathcal{W} \sqsubseteq_\lambda R$. Source executions modulo $\equiv_\partial$ are then order-isomorphic to* $\mathsf{BRuns}(R)$*, preserving outcomes, order, effect identities,* fresh/alias*, the safety answer, and the most permissive monitor.* CheckReg *rejects before runtime startup if a required outcome is missing, effect identities are inconsistent, or a state-changing edge is not mediated.*
2) ***Exact checking and construction.*** *For $\Theta = \Theta_S(u)$, the ideal and compiled machines return the same one of three answers for the same source snapshot. An undefined derivation gives a verified, state-preserving* Invalid*. A unique δ with $\widehat{B}^\dagger_\delta = \varnothing$ gives a verified, state-preserving* Reject *and no installed realization. Otherwise, let $Y =$ $\mathsf{Compile}(\Theta)$ and c^+_δ be the checked record. In this positive case, the following conditions are equivalent: (1) $\widehat{B}^\dagger_\delta \neq \varnothing$, (2) an installed realization exists, (3)* $\mathsf{Ready}(S, u, Y)$ *holds, and (4) an* $\mathsf{Edge}(u, [\mathcal{H}^+_u]_{\cong})$ *successor satisfies* $\mathsf{AgentSec}(S^+; c^+_\delta, \epsilon)$*. The largest execution family and its monitor satisfy* Preserve*. The installed edit retains evidence for* $\mathsf{Completed}_{H,u}$ *and records* $\mathsf{Removed}_{H,u}$*. The three answer classes are mutually exclusive and exhaustive for that source snapshot.*
3) ***All four state components are necessary.*** *The full information view is sufficient, and discarding any component is insufficient:*

$$\mathsf{Exact}(\pi_J) \iff J = \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}.$$

Every exact information map distinguishes every separating state pair in table II. Maps that erase call-to-effect or receipt-to-effect identity and TargetOnly *are inexact. Generalized nonblocking can serve as an exact solver only after the trusted controller derives the workflow and outcome-specific completion conditions.*
4) ***Repeated use.*** *Every enabled fully parameterized event induces the corresponding partial derivative on the private-name equivalence classes of* Sec *and preserves* AgentSec*. Therefore, the theorem applies again after every finite reachable sequence of execution edits, registered extensions, protected calls, receipt-log updates, installations, and crashes. Each source snapshot is finite, but there is no uniform bound on execution length.*
5) ***Runtime correctness.*** *$\mathcal{R}$ is a divergence-insensitive weak bisimulation under* $\mathsf{obs}_{\mathrm{api}}$*, so every good pair satisfies $S_0 \approx^{\mathsf{di}}_{\mathcal{O}_{\mathrm{api}}} I_0$. The observation includes edit answers*

tied to the authenticated source snapshot, replacement authorization tokens, and return values. It excludes time, fairness, and networking after Dispatch. *Disjoint domains compose asynchronously.*

Proof sketch. Schema preservation and finite-checker reflection prove Clause 1. Fixed-point reasoning proves Clause 2, separating state pairs prove Clause 3, and event induction together with durable simulation proves Clauses 4 and 5. The appendix supplies the witnesses and durable simulation. □

VII. Executable Validation

a) Proof coverage: The executable validation checks the finite decision procedure and its correspondence to the formal development. Five Lean modules pass `--trust=0`, with their theorem mapping listed in section A-C. The factor bounds, pomset lifting, and bisimulation remain mathematical proofs rather than mechanized Lean results.

b) Executable checks: All 81 decision-procedure tests pass. The performance script runs each sample in a fresh Python process and reports the median of five repetitions on a 16-vCPU AMD EPYC 7R12 with CPython 3.12. For accepted instances with $2, 8, 32, 64$, and 128 outcomes, checker time grows from 0.11 to 53.47 ms. The corresponding rejected instances take 0.11 to 5.51 ms. Enumerating $6, 24, 120$, and 720 unordered completions takes 0.33 to 50.35 ms. These timings cover only the checker core and exclude workload construction, edit derivation, installation, dispatch, and protected-call mediation. The executable caps are 128 outcomes, eight occurrences per outcome, 50,000 linearizations per outcome, and 100,000 indexed completed executions; larger instances return ResourceLimit and install nothing.

VIII. Related Work

a) Recovery and rollback: Output-commit recovery delays effects until state commits and reconstructs a fixed computation after failure. It does not decide whether an Agent may safely change its branch and checkpoint structure while several outcomes share one-use permissions [20], [21].

b) Supervisory control and synthesis: Generalized nonblocking starts from a supplied transition system, called the plant, and supplied completion conditions, called markings. Live Synthesis similarly starts from an old specification and an execution [15], [16], [17]. Our checker instead derives the edited workflow, every required outcome, and the necessary-and-sufficient safety condition for all six edits from the authenticated execution record before installation.

c) Agent recovery mechanisms: ACRFence blocks replayed effects after Restore, and DART selects checkpoints that preserve earlier work. Atomix delays settlement until its required branch boundary, while Rebound makes rollback atomic and auditable [22], [14], [12], [23]. These mechanisms take logs, scopes, call classes, branch boundaries, or policies as inputs rather than deriving the full safety condition of an execution edit from its authenticated record.

IX. Conclusion

Agent runtimes that Fork, Restore, or Merge an execution must account for both the workflow they change and calls that have already been authorized. Our checker derives the safety answer from a validated workflow, its authenticated execution record, the receipt log, and the requested edit. When the edit is safe, the greatest fixed point yields the most permissive monitor. When it is unsafe, the empty fixed point yields a finite proof that no implementation can satisfy the requirements. The Exact Edit-Safety theorem connects this decision to atomic installation and identifies the four kinds of recorded information required by any exact checker. Every supported event preserves AgentSec, so the theorem applies again after every finite reachable sequence of edits, calls, crashes, and registered extensions. The Lean modules and executable tests validate the finite decision procedure and its implementation.

APPENDIX A
SUPPLEMENTARY PROOF APPENDIX

This appendix supplies the proof objects used by theorem 1 for one finite registered call model and one atomic policy-domain monitor version at a time. Independent domains retain the product interpretation stated in the paper.

A. The Declarative Specification and Its Computed Witness

Fix a well-formed safety-check instance $\Theta = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, u)$ for which the edit derivation yields $\delta = (\kappa_u, H_u, \mathcal{A}_u, \mathcal{C}_u, \eta^-, \eta^+)$. All sets in this subsection are indexed sets, so two equal resolved words belonging to different outcomes remain different elements.

Definition 5 (Required-outcome preservation)**.**

$$\mathsf{Preserve}(H, u, \widehat{B}) \iff \forall a \in \mathsf{Req}_u.\ \exists t \in O_{H,u}, b.\ \mathsf{Lift}_u(t, a) \wedge (t, b) \in \widehat{B}.$$

Here $\mathsf{Removed}_{H,u}$ *contains the typed unselected-arm outcomes for* MergeSelect *or the current-role outcomes of a pending certified* RestoreReplace. $\mathsf{Completed}_{H,u}$ *contains the completed current role of* RestoreReplace. *Both sets are empty for every other row and for a registered extension. Thus* $\mathsf{Req}_u = \mathsf{SourceReq}(H, u) \setminus (\mathsf{Completed}_{H,u} \uplus \mathsf{Removed}_{H,u})$, *so* Preserve *covers exactly the source outcomes that are neither completed nor authorized for removal.*

Let $\mathfrak{R}_\Theta$ be the finite set of indexed realizations from definition 3, ordered componentwise by

$$(L_1, \widehat{B}_1) \sqsubseteq (L_2, \widehat{B}_2) \quad \iff \quad L_1 \subseteq L_2 \ \wedge\ \widehat{B}_1 \subseteq \widehat{B}_2.$$

Lemma 9 (Declarative–algorithmic bridge)**.** *The declarative predicate* $\mathsf{GReal}(\Theta; L, \widehat{B})$ *has a witness if and only if* $\widehat{B}^\dagger_{H,u} \neq \varnothing$. *When a witness exists, it is unique and equals* $(W^\dagger_{H,u}, \widehat{B}^\dagger_{H,u})$. *This equality is a theorem relating two independently stated objects, rather than the definition of the ideal machine.*

Proof. Let $(L, \widehat{B}) \in \mathfrak{R}_\Theta$. Source and target fidelity and policy safety give $\widehat{B} \subseteq \widehat{B}_0$. For every $(o, b) \in \widehat{B}$, every prefix $z \preceq_{\mathrm{pref}} b$ belongs to $L = \mathsf{Pref}(\pi_2 \widehat{B})$. Persistent outcome coverage therefore supplies, for every $v \in \mathsf{Compat}(z)$, a pair $(v, b') \in \widehat{B}$ extending z. Thus $\widehat{B} \subseteq \widehat{\Phi}(\widehat{B})$. By monotonicity on the finite lattice $2^{\widehat{B}_0}$, every post-fixed set is contained in the greatest fixed point, so $\widehat{B} \subseteq \widehat{B}^\dagger$. Taking projected prefixes yields $L \subseteq W^\dagger$.

Conversely, suppose $\widehat{B}^\dagger \neq \varnothing$. The fixed-point equality supplies persistent coverage and, at ϵ, composes with checked lifts to establish Preserve; inclusion in $\widehat{B}_0$ supplies target fidelity and safe completed words, prefix closure of $\mathcal{A}$ supplies safety for every generated prefix, and $W^\dagger = \mathsf{Pref}(\pi_2 \widehat{B}^\dagger)$ supplies completion nonblocking. Hence $(W^\dagger, \widehat{B}^\dagger) \in \mathfrak{R}_\Theta$, and the preceding containment makes it greatest. If two greatest elements existed, each would contain the other componentwise, so they would be equal. If $\widehat{B}^\dagger = \varnothing$, the first half places every hypothetical realization inside the empty set, contradicting its required nonemptiness. □

Corollary 1 (Exact compilation boundary)**.** *The ideal edit transition is enabled by the declarative* $\mathsf{Spec}(\Theta)$ *exactly when the checked compiler returns a nonempty fixed point. On that edge, the ideal language* L^* *equals the generated language of* $\mathsf{Can}(W^\dagger, B^\dagger)$.

Proof. The first statement is lemma 9. The second combines that lemma with lemma 7. □

Proposition 2 (Output-sensitive compilation)**.** *Fix the unit-cost symbolic-checker model with constant-time finite-map lookup and policy-automaton transitions. Let*

$$D = |\Theta|_{\mathrm{chk}}, \qquad n = \max_o |X_{p_o}|,$$
$$\Lambda = \sum_o \sum_{w \in \mathsf{Lin}(p_o)} (|w| + 1), \quad N_B = |\widehat{B}_0|,$$
$$V_B = \sum_{(o,b) \in \widehat{B}_0} (|b| + 1).$$

$$\mathcal{G}_{\mathrm{cov}} = \{((o, b), z, v) \mid (o, b) \in \widehat{B}_0,\ z \preceq b,\ v \in \mathsf{Compat}(z)\}.$$

Here D *is the authenticated input size,* Λ *is indexed linearization volume, and* $\mathcal{G}_{\mathrm{cov}}$ *is the materialized prefix–required-outcome relation. The semantic compiler runs in* $O(D + n\Lambda + |\mathcal{G}_{\mathrm{cov}}| + V_B \log(2 + V_B))$ *time and* $O(D + \Lambda + |\mathcal{G}_{\mathrm{cov}}|)$ *space, including its emitted witnesses. Moreover,* $N_B \leq \sum_o |\mathsf{Lin}(p_o)|$, $|\mathcal{G}_{\mathrm{cov}}| \leq N_B(n+1)|O_{\mathcal{C}_u}|$, *and the number of nonempty strict-removal ranks is at most* N_B.

Proof. A backtracking topological-order enumerator emits all indexed linearizations in $O(n\Lambda)$ time, while resolution, policy simulation, and construction of raw and resolved prefix tries are linear in emitted volume. For every support key (z, v), maintain the number of remaining pairs (v, b') extending z, initially enqueue candidates having a required outcome with zero support, and remove candidates in synchronous batches. When removal makes a support count zero, enqueue exactly the candidates incident to that requirement for the next batch. Each candidate and each incidence in $\mathcal{G}_{\mathrm{cov}}$ is processed at most once, and the batches are exactly $\widehat{B}_i \setminus \widehat{B}_{i+1}$. Store one rank and cause per removed candidate rather than copied sets, and construct $\mathsf{Can}(W, B)$ by bottom-up sorting of finite-trie derivative signatures in $O(V_B \log(2 + V_B))$ time. □

B. Exact Certificate Payloads

Let enc be canonical and length-delimited, with κ authenticating $\mathcal{A}$. The symbolic LTS uses domain-separated injective authenticated constructors $\mathsf{Seal}_{\mathrm{prog}}$ and $\mathsf{Seal}_{\mathrm{cut}}$, so equal seals have equal payloads. For $\Theta = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A}, u)$, define

$$\gamma(\Theta) = \mathsf{Seal}_{\mathrm{cut}}(\mathsf{enc}(\mathsf{request}, \omega, \zeta, \kappa, H, \Delta, \mathsf{out}, \mathcal{A}, u)),$$
$$H_{\mathsf{prog}} = \mathsf{Seal}_{\mathrm{prog}}(\mathsf{enc}(\mathsf{prog}_Z, \widehat{B}^\dagger_{H,u})).$$

The source-snapshot seal is

$$\mathsf{cut}_Z = \mathsf{Seal}_{\mathrm{cut}}(\mathsf{enc}(\omega, \zeta, \kappa, H, \Delta, \mathsf{out}, u, \mathcal{C}_{H,u}, H_{\mathsf{prog}}, \eta^-, \eta^+)). \tag{12}$$

Because H contains χ, ν, ρ, β, this seal binds workflow progress, record version, registry state, monitor-entry/version

bindings, the receipt log, and the dispatch queue. The verifier re-derives the schema judgment and complete indexed fixed point, reconstructs $(W^\dagger, B^\dagger)$, verifies both seals, and checks the full automaton isomorphism

$$\mathsf{prog}_Z \cong \mathsf{Can}(W^\dagger, B^\dagger),$$

including its initial state, marking, and every labeled transition. Canonical encoding order makes Invalid, Reject, and Installable unique and equivariant. A byte implementation may use signatures or collision-resistant digests over enc, yielding computational refinement up to forgery or collision, while exact bisimulation uses the symbolic constructors.

Proof of lemma 6. An authenticated extension preserves the current workflow exactly, preserves effect-identity resolution and all indexed outcomes, and satisfies $\mathcal{A} \subseteq \mathcal{A}^+$, so every old safe completion and coverage witness remains valid for the successor operator. The current residual family is therefore post-fixed, and greatestness gives its inclusion in a nonempty successor fixed point. □

C. Lean Theorem Map

`RegistrationRefinement.checkRegistration_iff`, `RegistrationRefinement.compiled_cell_eq_iff_canonical_eq`, `RegistrationRefinement.compiled_mediation_exhaustive`, the three receipt theorems, and `RegistrationRefinement.compiled_admission_preserved` cover registration reflection and its link to the safety decision. `FiniteCore.compiler_eq_semantic` and the greatest-realization theorems cover the fixed point, while `HistoryStructure.deriveEdit_iff` covers all six edit derivations. `CompilationBridge.compiler_admit_exists_secure_install` constructs a valid installed state from an accepted edit, and its converse declarations recover acceptance from any valid install candidate. `OperationalSemantics.kernelTrace_preserves_agentSec`, `OperationalSemantics.live_extension_preserves_agentSec`, `OperationalSemantics.fresh_before_live_extension_stale`, `OperationalSemantics.alias_before_live_extension_stale`, and `OperationalSemantics.live_extension_before_old_use_denied` cover installed finite-prefix closure and the extension races. The audit module imports these declarations and prints only the project-allowed axioms `propext`, `Classical.choice`, and `Quot.sound`.

D. Normalized Evidence and Safe Projection

This subsection asks which parts of the authenticated security state are necessary to reproduce every safety-check and installation answer. We first normalize private identifiers so that representation choices cannot distinguish two states, then remove one information factor at a time and construct queries whose correct answers differ.

We make the normalization implicit in Sec explicit. Let $\mathcal{Q}_{\mathrm{sec}}$ pair every valid $V = \mathsf{Sec}(S; c, r)$ with a well-sorted $q ::= \mathsf{Decide}(u) \mid \mathsf{TryInstall}(u, Y)$, permitting stale names. Ans returns $\mathsf{Compile}(\Theta_V(u))$ for Decide, and for TryInstall returns the successor bundle exactly when $\mathsf{Ready}_V(u, Y)$, otherwise NoCommit. An equivariant information map f is exact iff some D_f maps every $\langle f(V), q\rangle_\cong$ to $[\mathsf{Ans}(V, q)]_\cong$. Let $\mathcal{K}$ be the finite typed key universe at a checked source snapshot, containing tagged outcome, occurrence, effect-identity, receipt, checkpoint, group, domain, monitor-version, policy-version, and schema-version keys. Public keys are authority labels and identifiers declared external by the interface, while every other key may be consistently renamed. For a typed query q, $\mathsf{supp}(q)$ is defined recursively over its full syntax, including all identifiers nested inside a supplied symbolic certificate, monitor, or source-snapshot seal. Let $\mathcal{F}$ be the finite set of typed field paths and assign every semantic base field a unique owner

$$\mathsf{own} : \mathcal{F}_{\mathrm{base}} \longrightarrow \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}.$$

$\mathbf{P}$ owns workflow constructor tags, current/checkpoint/target contract nodes, order and outcome membership, schema kind and source fingerprints, lift relations, and authorized-removal declarations. $\mathbf{I}$ owns occurrence–effect-identity incidence, effect-identity–normalized-request/label/scope rows, occurrence lineage, and logical monitor-entry and authorization-token slots. $\mathbf{E}$ owns the global resolved trace, current and checkpoint cursors, receipt–effect-identity/creator and retry paths, durable return records, ordered dispatch-queue phase rows, and remaining policy. $\mathbf{C}$ owns the domain, policy/schema/view versions, private namespace, monitor-version allocation and phase, current monitor-entry ownership, certificate origin and coordinates, and checkpoint snapshot coordinates. These four lists are disjoint and exhaustive for semantic base fields. Certificate bytes, seals, fingerprints, digests, derived permission traces, and foreign-key paths are derived fields rather than additional semantic facts.

Write $f \rightsquigarrow g$ when the value or well-typed interpretation of derived field g depends immediately on field f. The dependency graph is acyclic because canonical encodings are constructed from base relations in a fixed order. Every foreign-key path depends on its target. Authenticated monitor-entry and authorization-token bytes depend on their $\mathbf{I}$ slot and $\mathbf{C}$ monitor version and owner. Receipt reconstruction and the permission trace depend on the $\mathbf{E}$ receipt and cursor rows together with their $\mathbf{I}$ effect identities and labels. Target certificates, source-snapshot seals, and digests depend on every encoded base field in their payload. A normalized full view V is a compatible finite assignment to all base fields together with the unique values of all derivable fields. Compatibility requires equal values on shared typed keys and the well-formedness conditions of definition 1. The natural join $\mathbf{P} \bowtie \mathbf{I} \bowtie \mathbf{E} \bowtie \mathbf{C}$ is the compatible union of these uniquely owned base fields followed by deterministic recomputation of derived fields. It is undefined precisely when shared typed keys disagree or a required foreign-key target is absent.

For $J \subseteq \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}$, start with base fields owned by J and the public sort declarations needed to type the retained records. Repeatedly delete any foreign-key path whose target has been deleted and any derived field having a deleted transitive dependency. The unique fixed point of this deletion is $\mathsf{Ret}(J)$, and $\pi_J(V)$ is V restricted to $\mathsf{Ret}(J)$. This definition prevents an erased fact from surviving inside a seal, digest, archived receipt-only registry path, or certificate payload.

The incidence-erasure projection $\pi_{\neg oc}$ starts from the full view, deletes the occurrence–effect-identity relation, and applies the same dependency deletion while retaining the occurrence and effect-identity marginals. The projection $\pi_{\neg rc}$ analogously deletes receipt–effect-identity incidence and every path that reconstructs it, including receipt–creator, cursor–receipt, archived creator–effect-identity paths, and every degree, count, or adjacency summary derived from those paths, while retaining the receipt, creator, occurrence, and effect-identity marginals. State views are compared modulo a sort-preserving bijection on private keys that fixes all public keys. For possibly different literal queries q, q', write $(V, q) \cong (V', q')$ when one such bijection maps the full syntax of q to that of q'; equivalently, $\langle V, q\rangle_{\cong} = \langle V', q'\rangle_{\cong}$.

Lemma 10 (Projection is well defined)**.** *Natural join, dependency deletion, and private renaming commute. Consequently,* $[\pi_J(V)]_{\cong}$, $[\pi_{\neg oc}(V)]_{\cong}$, *and* $[\pi_{\neg rc}(V)]_{\cong}$ *do not depend on a representative or on a certificate serialization.*

Proof. A sort-preserving bijection preserves key equality, foreign-key reachability, ownership, and the dependency relation. It therefore maps each deletion round to the corresponding deletion round in the renamed view. Induction on the finite acyclic dependency graph proves that the retained fixed points correspond. Canonical recomposition is equivariant because it is a typed constructor over exactly those retained fields. □

Lemma 11 (Representation-independent observational separation)**.** *Let* V_0, V_1 *be finite valid full views and let* q_0, q_1 *be typed queries. If* $[\mathsf{Ans}(V_0, q_0)]_{\cong} \neq [\mathsf{Ans}(V_1, q_1)]_{\cong}$, *then every exact information map* f *distinguishes the joint inputs:* $(f(V_0), q_0) \ncong (f(V_1), q_1)$. *The claim applies to arbitrary encodings; it is not a claim about a minimal table layout or a unique representation.*

Proof. Assume toward contradiction that $(f(V_0), q_0) \cong (f(V_1), q_1)$. The joint decoder inputs $\langle f(V_0), q_0\rangle_{\cong}$ and $\langle f(V_1), q_1\rangle_{\cong}$ then coincide. Exactness forces equal answer orbits, contradicting the premise. □

E. Bootstrap States and Explicit Separation Pairs

Definition 6 (Checked registration refinement)**.** *A boundary-finite typed Agent workflow* $\mathcal{W}$ *is a finite-state outcome-labeled LTS whose complete paths end at outcome terminals and whose* τ*-erased protected-call projections* $\partial e = (o, s_1 \cdots s_n)$ *form finitely many classes under equality* $\equiv_\partial$. CheckReg *rejects any reachable and terminal-co-reachable strongly connected component containing a protected-call edge, permits erased* τ*-cycles, and then enumerates the finite classes.* R *stores a candidate map* λ, *and* $\mathsf{BRuns}(R)$ *is the finite outcome-indexed language obtained by linearizing its registered root pomsets and labeling every occurrence by its protected* Use. $\mathcal{W} \sqsubseteq_\lambda R$ *means that* λ *induces a bijection from boundary classes to* $\mathsf{BRuns}(R)$ *that preserves outcomes, incidence, and order. The check also requires effect identities to be exactly the equivalence classes of normalized requests, requires* λ *to be total, unique, and free of raw instructions, and requires receipt evolution to commute with* Resolve *without dropping a required outcome, call site, or receipt.*

Erased	Full-view pair	Exact answers
$u_R(\mathcal{C}) := \mathsf{RestoreReplace}(b, k, \sigma)$ with $\mathcal{C}_k = \mathcal{C}$.		
$\mathbf{P}$	$q^i_{\mathbf{P}} = \mathsf{MergeSelect}(g_i, L, \sigma)$; $T^0 = b_x{:}X \oplus^{\mathrm{open}}_{g_0} b_y{:}Y$, $T^1 = b_x{:}X \parallel_{g_1} b_y{:}Y$; joint inputs are isomorphic after erasure.	V^0: realize; V^1: Invalid (wrong mode).
$\mathbf{I}$	$q_{\mathbf{I}} = \mathsf{RestoreLive}(b_L, k, \sigma_{\mathbf{I}})$ after safe empty-root/ForkChoice/Save; $\Delta = \epsilon, \mathcal{A} = \mathsf{Pref}\{a\}$; $q^0_{\mathrm{cell}}(x) = q^0_{\mathrm{cell}}(y) = d_0$; $q^1_{\mathrm{cell}}(x) = d_0 \neq d_1 = q^1_{\mathrm{cell}}(y)$; $\ell(d_0) = \ell(d_1) = a$.	$(X \otimes Y_k) \oplus Y$: one fresh, accept; two fresh reject aa.
$\mathbf{E}$	$q_{\mathbf{E}} = \mathsf{TryInstall}(u_R(x), Y_0)$; Y_0 is compiled at V^0 after safe empty-root/ForkChoice/Save/fresh; $V^0 \xrightarrow{\mathsf{Dispatch}(d)} V^1$, changing only the release phase.	V^0: install; V^1: NoCommit (source snapshot changed).
$\pi_{\neg rc}$	$q_{\neg rc} = u_R(x)$ after safe empty-root/ForkChoice/Save/fresh; $q_{\mathrm{cell}}(x) = d$, $\mathcal{A} = \mathsf{Pref}\{a\}$; same marginals $\{c, x\}, \{p\}, \{d, e\}$; $(q_{\mathrm{cell}}(c), p) = (d, d)$ in V^0, (e, e) in V^1.	$\mathsf{alias}(x_k, d)$: accept; $\mathsf{fresh}(x_k, d)$: reject aa.
$\mathbf{C}$	$q_{\mathbf{C}} = \mathsf{TryInstall}(u_0, Y_0)$, with full package Y_0 compiled at V^0, is literal in both; name supplies agree off the monitor-version sort, while current monitor version, ownership, and dependent seals differ.	V^0: install; V^1: NoCommit.

TABLE II
FACTOR- AND INCIDENCE-ERASURE PAIRS WITH ISOMORPHIC RETAINED VIEWS AND OPPOSITE EXACT ANSWERS.

Every separation proof follows the same recipe. Starting from a common checked initial state, we vary exactly one security-state factor, keep the visible projection and literal query fixed, and obtain opposite correct answers. The resulting pair proves that an exact abstraction cannot erase that factor.

We construct the finite lower-bound witnesses from one literal empty initial state $\mathsf{S}_\emptyset$. A registered call model R compiled from a boundary-finite typed Agent workflow $\mathcal{W}$ contains only finite contracts, schemas, policy, the checked registration map, and authenticated occurrence, effect-identity, normalized-request, lineage, scope, and label rows. It contains no receipt, cursor, dispatch-queue entry, monitor version, certificate, return record, monitor state, or installation record. The registration relation has a step $\mathsf{S}_\emptyset \xrightarrow{\mathsf{register}(\mathcal{W}, R, n)} \mathsf{Reg}(R, n)$ exactly when $\mathsf{CheckReg}(\mathcal{W}, R)$ accepts and n is a fresh private namespace, followed by checked root installation. Root installation constructs empty receipt, cursor, and dispatch-queue state, sets $\nu = 0$, activates one monitor version, installs the canonical monitor, and authenticates every current binding.

Lemma 12 (Registration reflection)**.** *For finite-state $\mathcal{W}$ and finite R,* $\mathsf{CheckReg}(\mathcal{W}, R)$ *accepts iff the registered λ witnesses $\mathcal{W} \sqsubseteq_\lambda R$ in definition 6. On acceptance, the λ-lowerings of source executions modulo $\equiv_\partial$ correspond bijectively to* $\mathsf{BRuns}(R)$*. The correspondence preserves indexed outcomes, causal order, canonical invocation quotienting, receipt evolution, and the safety-check answer.*

Proof. The productive-SCC test terminates because the state graph is finite. A productive SCC containing a durable-site edge can be pumped before an outcome, producing projected words of unbounded length. Conversely, if no productive SCC contains such an edge, every productive SCC is boundary-silent, and contracting them yields a DAG with a finite enumerable projected language. The checker compares that language with $\mathsf{BRuns}(R)$ and directly decides outcome/site incidence, order, effect-identity attestation, total unique registration, raw-bypass exclusion, and prefix receipt commutation. Therefore it accepts exactly the semantic conjunction defining $\mathcal{W} \sqsubseteq_\lambda R$, rather than an oracle restatement. Boundary projection erases only internal steps, so matching projected paths induce the quotient-execution bijection. Canonical invocation equality and deterministic $\mathsf{Resolve}$ make receipt evolution commute with this bijection at every prefix. The realization families are therefore order-isomorphic, so the fixed-point operator preserves the safety decision, greatestness, and pullback of the monitor. □

Lemma 13 (Natural bootstrap)**.** *If* $\mathsf{CheckReg}(\mathcal{W}, R)$ *accepts and R's registered root contract and policy have a nonempty greatest realization, registration and checked root installation reach a finite state S_R with* $\mathsf{AgentSec}(S_R; c_R, \epsilon)$*. The installation record originates at the checked root derivation, while every later replacement records an execution-edit derivation or a registered extension.*

Proof. Registration contributes only the checked R and n, and deterministic allocation constructs every runtime identifier from them. The root judgment and checked lifts establish core/schema coherence, and the recomputed nonempty fixed point establishes required-outcome coherence. Empty runtime progress establishes execution coherence, the canonical program at q_ϵ establishes monitor coherence, and the atomic phase/binding assignments establish monitor-version coherence. Only initial installation creates a root origin, and every successful later installation stores its checked execution-edit or registered-extension derivation. □

The following finite pairs use private names except for each displayed query and permission labels. Each displayed state is reached from $\mathsf{S}_\emptyset$ by the stated registration, root-installation, Save, protected-use, and edit steps; omitted fields are identical up to the stated private renaming and dependency deletion.

a) $\mathbf{P}$*: workflow structure:* Let X and Y be singleton contracts with distinct occurrences x and y, a shared effect identity, and a universal residual policy. From the same registered singleton root, checked initial installation followed by $\mathsf{ForkChoice}$ reaches $V^0_{\mathbf{P}}$ with private group g_0, while the corresponding $\mathsf{ForkParallel}$ trace reaches $V^1_{\mathbf{P}}$ with private group g_1. The two views differ only in

$$V^0_{\mathbf{P}}.T = b_L{:}X \oplus^{\mathsf{open}}_{g_0} b_R{:}Y, \qquad V^1_{\mathbf{P}}.T = b_L{:}X \parallel_{g_1} b_R{:}Y.$$

Use the same registered $\mathsf{MergeSelect}$ schema with $q^i_{\mathbf{P}} = \mathsf{MergeSelect}(g_i, L, \sigma)$. The choice view has a unique edit derivation and a nonempty one-step greatest realization, whereas the parallel view has no $\mathsf{MergeSelect}$ derivation because its group mode is wrong. Deleting $\mathbf{P}$ removes the constructor tag and every authenticator depending on it; the private renaming $g_0 \mapsto g_1$ then makes the retained state–query pairs isomorphic.

b) Bootstrap convention for the incidence pairs: Let $\mathbf{1}$ be the singleton-outcome contract whose unique pomset is empty. Its completion ϵ gives a nonempty greatest realization under $\mathcal{A} = \mathsf{Pref}\{a\}$. Both pairs below install this safe root and then a checked $\mathsf{ForkChoice}$. Each inactive schema row needed to equalize effect-identity marginals is identical in both views and never becomes active.

c) $\mathbf{I}$*: occurrence–effect-identity incidence:* Let X, Y be singleton contracts with occurrences x, y, and let d_0, d_1 be effect identities without receipts, both labeled a. Two checked registered call models have identical topology, schemas, occurrence marginals, and effect-identity marginals but the following incidence:

	$q_{\mathrm{cell}}(x)$	$q_{\mathrm{cell}}(y)$
$V^0_{\mathbf{I}}$	d_0	d_0
$V^1_{\mathbf{I}}$	d_0	d_1.

From the installed $\mathbf{1}$ root, the common Fork schema derives $b_L{:}X \oplus^{\mathsf{open}}_g b_R{:}Y$. Each outcome uses one effect identity, so both Forks install under $\mathsf{Pref}\{a\}$. Save b_R before progress and issue the common query $q_{\mathbf{I}} = \mathsf{RestoreLive}(b_L, k, \sigma_{\mathbf{I}})$. Up to clone renaming, the target is $(X \otimes Y_k) \oplus Y$. In $V^0_{\mathbf{I}}$, the left outcome uses one fresh and one alias on d_0, while the bypass outcome uses one fresh, so every outcome has permission trace a. In $V^1_{\mathbf{I}}$, every left-outcome linearization uses fresh on both d_0, d_1, so its word is $aa \notin \mathcal{A}$. The left outcome is compatible with ϵ, so the first $\widehat{\Phi}$ round also removes the otherwise safe bypass completion and leaves the greatest fixed point empty. Erasing occurrence–effect-identity incidence and its dependent rows makes the retained views isomorphic while preserving equal marginals.

d) $\mathbf{E}$*: mutable effect progress:* Let C, X be singleton contracts with occurrences c, x, let $q_{\mathrm{cell}}(c) = q_{\mathrm{cell}}(x) = d$, and label d by a. From the safe $\mathbf{1}$ root, a checked Fork installs $b_L{:}C \oplus^{\mathsf{open}}_g b_R{:}X$; save b_R, then use c to complete the left arm, create receipt p, and prepare its dispatch-queue item. At the resulting $V^0_{\mathbf{E}}$, compile $u_{\mathbf{E}} = \mathsf{RestoreReplace}(b_L, k, \sigma_{\mathbf{E}})$ to the full package Y_0; its clone of x aliases d, so Y_0 is installable under $\mathcal{A} = \mathsf{Pref}\{a\}$. Let $V^0_{\mathbf{E}} \xrightarrow{\mathsf{Dispatch}(d)} V^1_{\mathbf{E}}$, which changes only $\mathbf{E}$'s dispatch-queue release phase and preserves $\mathbf{P}, \mathbf{I}, \mathbf{C}$ literally. For the common literal $q_{\mathbf{E}} = \mathsf{TryInstall}(u_{\mathbf{E}}, Y_0)$, $V^0_{\mathbf{E}}$ installs, whereas $V^1_{\mathbf{E}}$ returns $\mathsf{NoCommit}$ because the source-snapshot seal binds the queue phase.

e) $\pi_{\neg rc}$: receipt–effect-identity incidence: For a separate pair, keep the common active row $q_{\mathrm{cell}}(x) = d$, creator c, receipt p, and effect identities d, e labeled a, but set $q_{\mathrm{cell}}(c) = d$ and $p \mapsto d$ in $V^0_{\neg rc}$, versus $q_{\mathrm{cell}}(c) = e$ and $p \mapsto e$ in $V^1_{\neg rc}$. Both states follow the same safe empty-root/ForkChoice/Save/fresh shape above, have permission trace a, and use the common query $q_{\neg rc} = \mathsf{RestoreReplace}(b_L, k, \sigma_{\neg rc})$. The cloned x_k is $\mathsf{alias}(x_k, d)$ in $V^0_{\neg rc}$, but $\mathsf{fresh}(x_k, d)$ produces forbidden word aa in $V^1_{\neg rc}$. The two views have the same creator, receipt, occurrence, and effect-identity marginals and the same retained incidence $x \mapsto d$. $\pi_{\neg rc}$ deletes the differing archived row for c, the receipt incidence, and every derived reconstruction path, so the retained views are isomorphic. This proves the necessity of semantic receipt–effect-identity incidence, not of any particular receipt-table column.

f) $\mathbf{C}$: installation coordinates: Let a bootstrap namespace be a family $n = (n_s)_{s \in \mathsf{Sort}}$ of injective, per-sort fresh supplies. From the same literal empty initial state, register the same public call model R with supplies n^0, n^1 that agree on every non-monitor-version sort but mint distinct initial versions $\eta_0 \neq \eta_1$, and perform checked root installation. The resulting $V^0_{\mathbf{C}}, V^1_{\mathbf{C}}$ have literally identical $\mathbf{P}, \mathbf{I}, \mathbf{E}$ base fields, while $\mathbf{C}$'s active monitor version, monitor-entry ownership, certificate coordinates, and dependent authenticators differ. Compile the same registered edit u_0 at $V^0_{\mathbf{C}}$ to obtain $Y_0 = \mathsf{Installable}(\delta_0, W_0, \widehat{B}_0, Z_0)$, and submit the same literal query $q_{\mathbf{C}} = \mathsf{TryInstall}(u_0, Y_0)$ to both views. At $V^0_{\mathbf{C}}$, recomputation reproduces cut_{Z_0}, so installation succeeds. At $V^1_{\mathbf{C}}$, recomputation binds η_1 and its owner rather than the η_0-coordinates sealed in Z_0, so installation returns $\mathsf{NoCommit}$. Erasing $\mathbf{C}$ recursively erases monitor-version-dependent entry/token rows, certificate coordinates, seals, and authenticators; the identity bijection on every retained sort then witnesses $V^0_{\mathbf{C}} \cong_{q_{\mathbf{C}}} V^1_{\mathbf{C}}$.

Proposition 3 (Explicit factor and incidence lower bounds)**.** *Every pair above consists of finite states reached from the common empty initial state and satisfying* $\mathsf{AgentSec}$*. Each pair has q-isomorphic indicated projections and opposite answer orbits for its common literal query. Therefore every proper factor projection, $\pi_{\neg oc}$, and $\pi_{\neg rc}$ is inexact. Every exact abstraction must distinguish all displayed pairs.*

Proof. Lemma 13 gives the common initial state and valid root installation, while the explicit traces above use only modeled Save, protected-use, and execution-edit transitions. For the incidence pairs, the empty root completes with ϵ, and each installed Fork arm has a one-fresh completion labeled a. Hence every displayed pre-query state is reachable under $\mathsf{Pref}\{a\}$. Open choice enables the Save steps, and the installed monitors enable each stated left-arm fresh before that arm becomes a completed, still-addressable continuation. For the $\mathbf{E}$ pair, Dispatch preserves $\mathbf{P}, \mathbf{I}, \mathbf{C}$ but changes a queue phase bound by Y_0's source-snapshot seal, giving install versus $\mathsf{NoCommit}$. The displayed schema and resolution calculations give the other opposite answers. Lemma 10 and the specified private-name bijections, each pointwise fixing the common query support, give q-relative projection equality. Any projection omitting at least one factor collapses the corresponding factor pair, while the two incidence erasures collapse their corresponding incidence pairs. Inexactness and the general information lower bound now follow from lemma 11. □

Assigning the same caller-proposed target transition system, completion conditions, and monitor version to the $\mathbf{P}$ pair makes $\mathsf{TargetOnly}$ equal while leaving the authenticated queries and opposite answers unchanged. Thus target-only pre-derivation state is not exact, although generalized nonblocking is exact as a backend after trusted derivation supplies the plant, outcome identities, and conditional markings.

F. Event Derivatives and Invariant Preservation

Write $A_1, \ldots, A_5$ for the five clauses of $\mathsf{AgentSec}$ in their stated order. The following matrix is exhaustive for durable transitions inside an installed registered call model. Initial registration occurs before the root safety check; later additive schema and policy growth uses the registered-extension transition listed below.

Lemma 14 (Representative-independent event derivative)**.** *The guarded update induces a partial function ∂ on joint state–event orbits $\langle V, e\rangle_{\cong}$. If $\mathsf{AgentSec}(S; c, r)$ and $S \xrightarrow{e} S'$, the witness in table III satisfies $\mathsf{AgentSec}(S'; c', r')$ and*

$$[\mathsf{Sec}(S'; c', r')]_{\cong} = \partial(\langle \mathsf{Sec}(S; c, r), e\rangle_{\cong}).$$

Proof. All rule guards use typed equality, membership, order, canonical constructors, and authenticated payload equality. These operations are equivariant under a simultaneous private-key renaming of the state and event payload. For an event e, let $G_e(V)$ be its rule guard and $\mathsf{U}_e(V)$ its successor update when that guard holds. For every such renaming π, $G_e(V) \iff G_{\pi e}(\pi V)$ and $\mathsf{U}_{\pi e}(\pi V) = \pi \mathsf{U}_e(V)$. Thus equal pointed orbits have the same definedness and the same successor orbit. This proves representative independence and functionality.

For fresh and alias, lemmas 3, 5 and 7 establish the first two rows of the matrix. If x lies at b, HStep appends a to b's cursor, while the same atomic protected-use update appends a to the global resolved-occurrence trace χ; residual associativity gives

$$(\Gamma_b / \mathsf{raw}(\chi_b))/x = \Gamma_b / \mathsf{raw}(\chi_b a).$$

Thus the updated leaf and any later Save remain synchronized with G. For a successful edit transition, lemmas 2 and 6 to 9 establish the third row for all six schema rows and registered extension. Each remaining rule changes only the fields shown in the matrix, and its row checks every invariant clause whose fields change. Thus every successor has the displayed witness and normalized update. □

Corollary 2 (Finite-prefix closure)**.** *Starting from any valid bootstrap, every finite prefix of any sequence of the event classes in table III ends in a state satisfying* $\mathsf{AgentSec}$*. The sequence length is unbounded, although every individual safety-check instance and event payload is finite. For a finite word $\bar{e}$, the*

Event class	Durable update	Witness	Preservation argument
fresh use	Apply HStep to (G, T); append $\mathsf{fresh}(x, d)$ to global χ; advance ν, q; append one receipt and normalized-request queue item; remove the current binding.	$(c, r\mathsf{fresh}(x, d))$	A_1: registry and schema unchanged. A_2: residual coherence. A_3: unique receipt and ordered preparation. A_4: canonical derivative. A_5: remove exactly x's binding.
alias use	Apply HStep to (G, T); append $\mathsf{alias}(x, d)$ to global χ; advance ν, q; add only the return-record link; remove current binding.	$(c, r\mathsf{alias}(x, d))$	A_1: authenticated effect identity retained. A_2: residual coherence. A_3: link names an earlier receipt and the permission trace is unchanged. A_4: canonical derivative. A_5: remove exactly x's binding.
Successful edit transition, six edits or registered extension	Install derived $(\kappa^+, H^+, \mathcal{A}^+)$, checked monitor and certificate; close η^-; activate η^+; publish new bindings.	(c^+_δ, ϵ)	A_1: schema fidelity and lifts. A_2: declarative–algorithmic bridge and checker. A_3: source receipts and dispatch queue retained. A_4: install at q_ϵ. A_5: complete monitor replacement.
Verified Invalid or Reject	Emit the certificate for the current source snapshot; no durable state mutation.	(c, r)	All clauses unchanged; γ binds the visible answer to this source snapshot.
Save	Snapshot the current residual/cursor pair in an authenticated checkpoint; advance ν.	(c, r)	A_1: immutable well-typed extension. A_2, A_4, A_5: unchanged. A_3: exactly the permitted K extension.
Preload	Allocate or replace content only in an inactive unbound monitor version.	(c, r)	A_1–A_4: unchanged. A_5: inactive and unbound remain true; no authorization token is exposed.
Retry, retrieval, delivery	Emit $\mathsf{Return}(t, [v]_{\cong})$ for an authenticated durable value; no security-state mutation.	(c, r)	All clauses unchanged; retry additionally checks the same canonical invocation and receipt link.
Denial	Return $\mathsf{deny}(x)$; no state mutation.	(c, r)	All clauses unchanged.
$\mathsf{Dispatch}(d)$	Mark exactly the oldest prepared dispatch-queue item released.	(c, r)	A_3: release order remains a prefix of preparation order. Other clauses unchanged.
Settlement	Emit $\mathsf{Settle}(d, [v]_{\cong})$, advance one receipt phase, and fill its stable return record.	(c, r)	A_3: effect identity, normalized request, creator, permission label, and order are immutable. Other clauses unchanged.
Stale or malformed candidate	No durable state mutation.	(c, r)	All clauses unchanged; the attempt is τ and emits no verdict.
Crash and recovery	Discard volatile work and expose the same recovered durable image.	(c, r)	All clauses unchanged under the assumed crash-stable transaction boundary.

TABLE III
EXHAUSTIVE EVENT CLASSES AND PRESERVATION OF THE FIVE AgentSec CLAUSES.

iterated derivative is defined on $\langle V, \bar{e}\rangle_{\cong}$, where one renaming acts on the initial view and the whole word.

Proof. The base case is lemma 13, and lemma 14 supplies both the invariant-preserving inductive step and equivariance under one renaming of the remaining event suffix. □

G. Uncontrollable Events and Recovered-State Crashes

The resolved alphabet contains only commits mediated by the protected monitor. Agent request arrival, scheduling, settlement, delivery, and crash are environmental events, but none appends a fresh or alias symbol. Request arrival and scheduler choice have no durable transition until a guarded rule commits. Settlement and delivery emit modeled API observations but leave the resolved prefix unchanged. Dispatch is observable only when the oldest prepared item is durably released, and it also leaves the resolved prefix unchanged. Therefore these uncontrollable events stutter with respect to the conditional-nonblocking plant state. A deployment in which an environmental action itself commits a resolved occurrence is outside this contract and must instead synthesize a supremal controllable sublanguage.

Lemma 15 (Uncontrollable stutter)**.** *If $\mathsf{AgentSec}(S; c, r)$, e is an uncontrollable modeled event, and $S \xrightarrow{e} S'$, then the resolved prefix and canonical monitor state are unchanged and $\mathsf{AgentSec}(S'; c, r)$ holds.*

Proof. Request arrival and scheduling have no durable transition, while denial, retry, retrieval, delivery, and crash preserve the recovered security state. Dispatch and settlement change only the monotone dispatch queue, receipt phase, or durable return record permitted by execution coherence. None changes $H.T$, $H.\chi$, the permission trace derived from Δ, r, or $\mathsf{q}(\eta)$, and the corresponding row of table III preserves every other invariant clause. □

The crash statement uses the recovered-state interface assumed in section III, not a new storage protocol. Let D be a concrete durable image, let v be volatile state, and let t be at most one in-flight domain transaction. Linearizability and crash stability provide a recovery function satisfying

$$\mathsf{Recover}(D, v, t) = \begin{cases} D, & t \text{ has not linearized,} \\ \mathsf{commit}_t(D), & t \text{ has linearized.} \end{cases}$$

No recovered image contains a strict subset of one modeled atomic update. In particular, a fresh transaction cannot expose a receipt without its global-trace record, branch-local cursor record, and dispatch-queue preparation. Installation cannot activate a new monitor version without closing the old version and installing all target bindings.

Lemma 16 (Recovered-state crash refinement)**.** *If a concrete execution refines compiled state S immediately before a transaction linearizes, recovery refines S. If the transaction linearizes to a rule successor S', recovery refines S'. At recovered-state LTS boundaries, crash is consequently a τ self-loop.*

Proof. The recovery equation selects the complete pre-transaction durable image or the complete committed post-transaction durable image. The former has the same abstraction as S, while the latter has the unique guarded update described by the corresponding row of table III and therefore abstracts to S'. Once the LTS records the recovered image, crashing again changes only volatile state and hence becomes a self-loop. □

H. Durable Agent-API Weak Bisimulation

We now prove the durable-realization clause of theorem 1 by exhausting the observable and silent cases. For $K = (\kappa, H, \Delta, \mathsf{out}, \mathcal{A})$, $\mathsf{Current}(H) = X_{[\![H.T]\!]}$, and $I = (K, c_I, r_I)$, let $\alpha_I(I) = (K, c_I, r_I, H.\eta, H.\beta)$ and $\alpha_S(S; c, r) = (K, \mathsf{icut}(c), r, H.\eta, \mathsf{bind}|_{\mathsf{Current}(H)})$. Define $I \mathcal{R} S$ iff some c, r satisfy $\mathsf{AgentSec}(S; c, r)$ and $\alpha_I(I) = \alpha_S(S; c, r)$; write $\overset{\ell}{\Longrightarrow}= \tau^* \ell \tau^*$ and $\overset{\tau}{\Longrightarrow}= \tau^*$. Fix $I \mathcal{R} S$ with witness (c, r). Equality of $\alpha_I(I)$ and $\alpha_S(S; c, r)$ gives both machines the same κ, current policy, normalized execution record, receipt log, dispatch queue, durable return records, ideal projection, current partial execution, monitor version, and monitor-entry bindings.

a) Edit transitions: The normalization equality makes both machines form the same current $\Theta_I(u)$ and source-snapshot token $\gamma(\Theta_I(u))$. If derivation is undefined, deterministic failed-check reflection makes both sides emit the same canonical Invalid orbit and take a state self-loop. If derivation exists but no declarative realization exists, lemma 9 and deterministic fixed-point checking make both sides emit the same canonical Reject orbit and take a state self-loop. Otherwise the ideal machine enables its edit transition and, by lemma 9, the compiled checker accepts exactly the same edit and installs the same largest language. The compiled side may take finitely many inactive-preloading τ-steps before atomic installation. Pure compilation is outside the recovered-state LTS. The ideal side takes its single atomic edge, and both successors normalize to the same $(\kappa_u, H_u, \mathcal{A}_u)$, ideal projection, empty prefix, active monitor version, and target bindings. Deterministic allocation gives the same authorization-token bundle up to private renaming, so both expose $\mathsf{Edge}(u, [\mathcal{H}_u^+]_{\cong})$. Conversely, every successful compiled installation has passed the schema, fixed-point, certificate, and source-snapshot checks, so lemma 9 enables the matching ideal edge. This argument applies uniformly to all six execution edits and the registered extension.

b) Protected uses and concrete invocations: For a submitted tool request m, both guards compute the same $m^\circ = \mathsf{canon}_\kappa(m)$. The compiled authorization-token check verifies the call-authorizing service, scope, occurrence, monitor entry, effect identity, lineage, signed digest, current monitor version, and equality $m^\circ = \mathsf{inv}_\rho(d)$. The ideal guard requires the same normalized binding and request equality. Both sides then inspect the same set D_Δ of effect identities with receipts, so they choose the same $a \in \{\mathsf{fresh}(x, d), \mathsf{alias}(x, d)\}$. By corollary 1 and lemma 7, ra is enabled by the compiled derivative exactly when $ra \in L^*$ in the ideal machine.

In the fresh case, both successors apply the same paired residual, append the same symbol to the global trace and branch-local cursor, advance ν, create the same effect-identity-bound receipt, and prepare the same normalized request. The compiled dispatch queue stores $(d, \mathsf{inv}_\rho(d))$, never the caller buffer, while the ideal queue stores (d, m°); the verified equality makes these records identical. In the alias case, both successors apply the same paired update, append the same receipt-free symbol to the global record and branch cursor, and bind the occurrence to the same durable return record without changing Δ or the dispatch queue. Both successors remove x's current binding; because the remaining workflow no longer contains x, their normalized current-binding components remain equal. In either case, lemma 14 re-establishes $\mathcal{R}$ with witness (c, ra).

If any occurrence, monitor entry, authorization token, normalized request, monitor version, branch, or monitor transition check fails, both normalized guards fail. Both machines emit $\mathsf{deny}(x)$, preserve state, and remain related.

c) Use–installation races: Use and installation serialize on the same execution and monitor-version records. If a fresh use commits first, it changes $(G, T, \Delta, \chi, \nu, \mathsf{out})$, all of which are bound directly or through H by the source-snapshot seal, so the prepared installation becomes stale. If an alias use commits first, it changes (G, T, χ, ν) even though Δ and the dispatch queue are unchanged, so the prepared installation again becomes stale. If installation commits first, it atomically closes η^-, activates η^+, refreshes every current monitor entry and authorization token, and exposes replacement tokens only after activation. The old fresh or alias use then fails the current-monitor-version guard and is denied without progress. The ideal atomic order has exactly the same two cases, so neither side admits a third race outcome.

d) Replies, releases, and silent steps: An authenticated retry, authorization-token retrieval, or result delivery leaves the security state unchanged and follows the same authenticated link on both sides to expose $\mathsf{Return}(t, [v]_{\cong})$. $\mathsf{Dispatch}(d)$ is enabled at the same dispatch-queue head and makes the same observable release update. $\mathsf{Settle}(d, [v]_{\cong})$ is enabled for the same receipt and produces the same stable result and normalized update. Save takes matching τ-steps. Compiled preload and stale or malformed candidate attempts are implementation-only τ-steps matched by zero ideal steps because α_S omits their inactive contents and they emit no verdict. Crash is matched by a τ self-loop on each recovered-state machine by lemma 16.

Theorem 2 (Durable Agent-API realization, detailed)**.** *$\mathcal{R}$ is a divergence-insensitive weak bisimulation under* $\mathsf{obs}_{\mathrm{api}}$.

Proof. The least-generated rule sets in sections IV-C and V and table III make the preceding cases exhaustive. For every step from either side, the corresponding case constructs a path with observation $\mathsf{obs}_{\mathrm{api}}(\ell)$ and a successor related by the witness in that matrix. The construction is symmetric because successful ideal guards and successful compiled guards were shown equivalent, while implementation-only steps preserve the common abstraction. Arbitrary repetition of silent preload, stale-candidate, Save, or crash steps is ignored, which is precisely divergence-insensitive weak matching. □

I. Executable-Artifact Bounds

Above 128 outcomes, eight occurrences per outcome, 50,000 linearizations, or 100,000 completions, the executable wrapper returns $\mathsf{ResourceLimit}(b, n)$ before search and performs no installation. This fail-closed engineering outcome is neither a

semantic rejection nor an impossibility certificate. The theorem uses the uncapped finite semantic compiler.

No step in this proof infers a natural-language contract, provides fairness, disables only part of one domain, or strengthens local queued-request release into remote exactly-once execution.

AI Use Acknowledgment

OpenAI Codex was used throughout the manuscript for initial prose drafting and revision, and in the executable artifact for code scaffolding and test generation. It also served as an interactive aid for literature discovery, counterexample search, and the discussion and refinement of the formal theory, including definitions, assumptions, theorem statements, and proof structure. The authors chose and directed the research, made all final theoretical and methodological decisions, independently verified the cited sources, theorem statements, proofs, code, and experimental results, and remain responsible for every claim, proof, result, and citation.

References

[1] OpenAI, "Codex App Server," Official documentation, 2026, accessed 2026-07-31. [Online]. Available: https://learn.chatgpt.com/docs/app-server.md

[2] Anthropic, "Manage sessions," Official documentation, 2026, accessed 2026-07-31. [Online]. Available: https://code.claude.com/docs/en/sessions

[3] ——, "Checkpointing," Official documentation, 2026, accessed 2026-07-31. [Online]. Available: https://code.claude.com/docs/en/checkpointing

[4] C. Wang and Y. Zheng, "Fork, explore, commit: Os primitives for agentic exploration," 2026, agentic OS Workshop 2026. [Online]. Available: https://arxiv.org/abs/2602.08199

[5] T. Wu, C. Chang, L. Cao, W. Gao, and W. Wang, "Crab: A semantics-aware checkpoint/restore runtime for agent sandboxes," 2026. [Online]. Available: https://arxiv.org/abs/2604.28138

[6] Y. Zhang, "Agent libos: A runtime substrate for capability-controlled self-evolving llm agents," 2026. [Online]. Available: https://arxiv.org/abs/2606.03895

[7] S. Yu, D. Chong, A. Nandi, D. Soylu, J. Sun, C. D. Manning, and W. Shi, "Shepherd: Enabling programmable meta-agents via reversible agentic execution traces," 2026. [Online]. Available: https://arxiv.org/abs/2605.10913

[8] Y. Li, S. Ping, X. Chen, X. Qi, Z. Wang, Y. Luo, and X. Zhang, "AgentGit: A version control framework for reliable and scalable LLM-powered multi-agent systems," 2025. [Online]. Available: https://arxiv.org/abs/2511.00628

[9] LangChain AI, "Use time-travel," 2026, accessed 2026-08-03. [Online]. Available: https://docs.langchain.com/oss/python/langgraph/use-time-travel

[10] S. G. Patil, T. Zhang, V. Fang, N. C., R. Huang, A. Hao, M. Casado, J. E. Gonzalez, R. A. Popa, and I. Stoica, "GoEX: Perspectives and designs towards a runtime for autonomous LLM applications," 2024. [Online]. Available: https://arxiv.org/abs/2404.06921

[11] E. Y. Chang and L. Geng, "SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning," *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 4874–4886, 2025. [Online]. Available: https://www.vldb.org/pvldb/vol18/p4874-chang.pdf

[12] B. Mohammadi, N. Potamitis, L. Klein, A. Arora, and L. Bindschaedler, "Atomix: Timely, transactional tool use for reliable agentic workflows," 2026. [Online]. Available: https://arxiv.org/abs/2602.14849

[13] Z. Chen, H. Liu, D. Xu, D. Dong, J. Li, B. Pu, and J. Zhai, "Cordon: Semantic transactions for tool-using llm agents," 2026. [Online]. Available: https://arxiv.org/abs/2606.17573

[14] K. Yang, P. Li, Z. Wu, K. Xu, H. Huang, and X. Huang, "DART: Semantic recoverability for structured tool agents," 2026. [Online]. Available: https://arxiv.org/abs/2605.23311

[15] M. H. de Queiroz, J. E. R. Cury, and W. M. Wonham, "Multitasking supervisory control of discrete-event systems," *Discrete Event Dynamic Systems*, vol. 15, no. 4, pp. 375–395, 2005.

[16] R. Malik and R. Leduc, "Generalised nonblocking," in *Proceedings of the 9th International Workshop on Discrete Event Systems*, 2008, pp. 340–345.

[17] B. Finkbeiner, F. Klein, and N. Metzger, "Live synthesis," *Innovations in Systems and Software Engineering*, vol. 18, no. 3, pp. 443–454, 2022. [Online]. Available: https://doi.org/10.1007/s11334-022-00447-5

[18] L. Aceto, I. Cassar, A. Francalanza, and A. Ingólfsdóttir, "On runtime enforcement via suppressions," in *29th International Conference on Concurrency Theory (CONCUR 2018)*, ser. Leibniz International Proceedings in Informatics, vol. 118. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018, pp. 34:1–34:17.

[19] J. A. Brzozowski, "Derivatives of regular expressions," *Journal of the ACM*, vol. 11, no. 4, pp. 481–494, 1964.

[20] R. E. Strom and S. Yemini, "Optimistic recovery in distributed systems," *ACM Transactions on Computer Systems*, vol. 3, no. 3, pp. 204–226, 1985.

[21] E. Y. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, "A survey of rollback-recovery protocols in message-passing systems," *ACM Computing Surveys*, vol. 34, no. 3, pp. 375–408, 2002.

[22] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn, "ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore," 2026, coDAIM Workshop 2026. [Online]. Available: https://arxiv.org/abs/2603.20625

[23] Q. Burke, A. Vahldiek-Oberwagner, M. Swift, and P. McDaniel, "It's a feature, not a bug: Secure and auditable state rollback for confidential cloud applications," in *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2026, preprint first posted in 2025. [Online]. Available: https://arxiv.org/abs/2511.13641